

# Taming Multi-Dimensional Skew in Sparse Matrix Multiplication with Contention-Aware Scheduling [Technical Report]

Wentao Huang  
National University of Singapore  
huangwentao@u.nus.edu

Qiming Huang  
National University of Singapore  
huang\_qiming@u.nus.edu

Yunhong Ji  
Renmin University of China  
jiyunhong@ruc.edu.cn

Mian Lu  
4Paradigm Inc.  
lumian@4paradigm.com

Yuqiang Chen  
4Paradigm Inc.  
chenyuqiang@4paradigm.com

Bingsheng He  
National University of Singapore  
dcsheb@nus.edu.sg

Kian-Lee Tan  
National University of Singapore  
dcstankl@nus.edu.sg

## ABSTRACT

Sparse general matrix-matrix multiplication (SpGEMM) is a key kernel in graph analytics, relational queries, and data mining. Yet, its performance remains a significant challenge, due to the inherent sparsity and, more critically, the prevalent skew patterns often found in real-world workloads. This paper investigates this problem and demonstrates that an unbalanced workload distribution is the main culprit, harming performance through two co-existing root causes: first, *multi-dimensional skew*, where imbalance across computation cost, output density, and data locality cannot be captured by any single balancing metric; and second, *cache contention*, where dense substructures induce severe on-chip interference.

To that end, we propose adaptive morsel scheduling (AMS), a dynamic scheduling framework that decouples skew dimensions via fixed-result-size morsels and integrates contention-aware load balancing (CALB) and data-conscious simultaneous multithreading activator (SMTact) to alleviate cache-level contention, alongside a hybrid accumulation mechanism (HYB) that adaptively selects the best accumulator per morsel. Evaluations across matrix squaring, relational queries, graph analytics, and other sparse matrix kernels show that AMS significantly outperforms state-of-the-art methods on skewed workloads while maintaining comparable performance on non-skewed ones. These gains, along with improved bandwidth utilization, remain valid across a wide range of workloads and different hardware systems.

## PVLDB Reference Format:

Wentao Huang, Qiming Huang, Yunhong Ji, Mian Lu, Yuqiang Chen, Bingsheng He, and Kian-Lee Tan. Taming Multi-Dimensional Skew in Sparse Matrix Multiplication with Contention-Aware Scheduling [Technical Report]. PVLDB, 14(1): XXX-XXX, 2020.  
doi:XX.XX/XXX.XX

## PVLDB Artifact Availability:

This work is licensed under the Creative Commons BY-NC-ND 4.0 International License. Visit <https://creativecommons.org/licenses/by-nc-nd/4.0/> to view a copy of this license. For any use beyond those covered by this license, obtain permission by emailing [info@vldb.org](mailto:info@vldb.org). Copyright is held by the owner/author(s). Publication rights licensed to the VLDB Endowment.  
Proceedings of the VLDB Endowment, Vol. 14, No. 1 ISSN 2150-8097.  
doi:XX.XX/XXX.XX

The source code, data, and/or other artifacts have been made available at <https://github.com/fukien/casmm>.

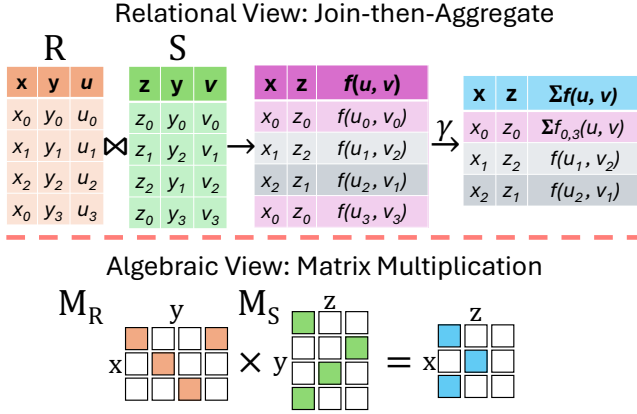
## 1 INTRODUCTION

Matrix multiplication is a fundamental computational kernel, and its sparse variant, sparse general matrix-matrix multiplication (SpGEMM), is especially important in practice. The reason is that sparse matrices are a natural model for real-world graph data, which makes SpGEMM a core building block for applications such as graph analytics, entity resolution, and machine learning [40, 62, 76, 79, 87, 104, 128]. Recently, SpGEMM has emerged as a transformative method for processing relational join-aggregate and join-project queries [33, 54–56, 59, 66, 101–103]<sup>1</sup>. By treating relational tables as algebraic sparse matrices, entire query pipelines can be executed as unified algebraic operations. This enables aggregation or projection to be performed *on-the-fly* alongside the join, effectively bypassing the prohibitive costs of materializing excessive intermediate results (see Figure 1 for a comparison). In all such cases, the performance of the application is dictated by the efficiency of the underlying parallel SpGEMM execution. Reflecting its critical role, numerous studies have investigated SpGEMM optimization, reaching a broad consensus that it is a fundamentally bandwidth-intensive kernel where throughput is ultimately governed by available memory bandwidth [47, 84].

Though it is bandwidth-bound, running SpGEMM on a high-bandwidth platform does not necessarily translate to higher throughput<sup>2</sup>. This is primarily attributed to the *load imbalance* caused by the widely observed skew patterns in real-world workloads [15, 29, 65, 71, 95, 113, 127]. For instance, in a typical OLAP ClickHouse benchmark [97], the top 5% of users generate 24.4% (47.8 million) of all clicks, while the median number of clicks per user is only 2. Similarly, in a common PageRank workload [56, 119], the top 1.8% of pages receive more incoming links than the remaining 98.2% combined. From a matrix perspective, such skewed key frequencies and power-law graph degrees translate into highly irregular nonzero

<sup>1</sup>We evaluate both query classes in § 4.6, as they share the same algebraic formulation and differ only in whether the value domain is referenced.

<sup>2</sup>While SpGEMM above a certain density threshold can be compute-limited, the growing compute-bandwidth gap [43, 73, 91, 109] makes compute progressively less likely than bandwidth to be the primary bottleneck.



**Figure 1: Comparing two approaches for evaluating a join-aggregate query, “SELECT x, z, SUM(f(u, v)) FROM R, S WHERE R.y = S.y GROUP BY x, z”, where f denotes a mathematical function. The “relational view” (top) joins tables R(x, y, u) and S(z, y, v) on attribute y followed by an aggregation, a process that produces excessive intermediate results, such as (x<sub>0</sub>, z<sub>0</sub>, f(u<sub>0</sub>, v<sub>0</sub>)) and (x<sub>0</sub>, z<sub>0</sub>, f(u<sub>3</sub>, v<sub>3</sub>)), which are ultimately aggregated into one final group-by result (shaded in light pink). The “algebraic view” (bottom) models the operation as a matrix product  $M_R \times M_S$  where y serves as the shared dimension to compute the aggregate result directly, bypassing the generation of costly intermediate results.**

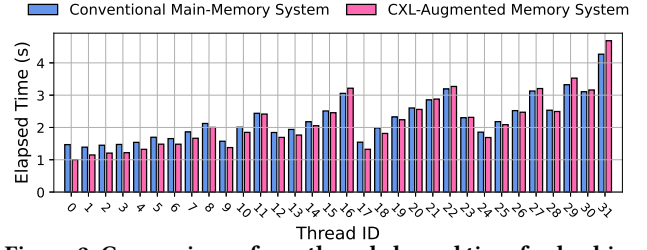
structures, resulting in significant load imbalance. When running SpGEMM on these workloads, a small subset of cores becomes overloaded while others remain underutilized [40, 47, 84]. This not only limits compute efficiency but also prevents full utilization of the available memory bandwidth, leaving system resources idle, and therefore, rendering high-bandwidth platforms less effective.

Moreover, the challenge of addressing load imbalance in skewed workloads is particularly critical and is further exacerbated by the “memory scaling wall,” where memory technology advancements lag behind those of processing units [91, 109]. This trend results in decreasing memory bandwidth and capacity per processing core [58, 117], which further compounds the load imbalance problem for two reasons:

First, decreasing bandwidth-per-core has an acutely negative impact on SpGEMM workloads with load imbalance, where a few straggling threads drag down performance. Since these threads largely rely on core-level bandwidth, their overall performance is inevitably reduced due to the insufficient bandwidth-per-core caused by the “memory scaling wall.”

Second, the prevalent approach to compensating for decreasing bandwidth-per-core, namely the adoption of heterogeneous memory expansion technologies like CXL [57, 70, 117, 118], can inadvertently amplify this problem. Since CXL-attached memory typically exhibits higher access latency than local DRAM<sup>3</sup>, straggling threads that run disproportionately long are more exposed

<sup>3</sup>Since DRAM is the primary technology used in memory manufacturing, we use the terms “DRAM” and “memory” interchangeably throughout this paper.



**Figure 2: Comparison of per-thread elapsed time for hashing-based SpGEMM [84] on a G500 skewed matrix (scale=17, edge-factor=16) [24] between a conventional main-memory system (28.50 GB/s bandwidth) and a CXL-augmented system (46.50 GB/s bandwidth) (see § 4 for experimental setups).**

to CXL’s latency penalty, further widening the gap between fast and slow threads. Figure 2 illustrates this using a hashing-based SpGEMM algorithm [84] on a skewed workload, where per-thread runtimes vary due to load imbalance.<sup>4</sup> Because total runtime is governed by the slowest thread, the disproportionate latency on stragglers negates the aggregate bandwidth gain, yielding worse overall performance than pure DRAM. This counterintuitive result is also reported in existing CXL studies [77, 78, 105, 108]. As a result, addressing load imbalance is becoming more imperative than ever.

However, addressing load imbalance in SpGEMM is not trivial. While numerous strategies have been proposed [40, 47, 88], we maintain that existing approaches fall short due to two co-existing root causes: *multi-dimensional skew* and *cache contention*, both of which not only hinder load balancing but also degrade overall performance.

First, existing methods fail to accommodate the *multi-dimensional skew* inherent in sparse matrices. Current methods rely on static schemes that balance along a single metric (e.g., flop count) to minimize scheduling overhead. However, skew manifests across multiple dimensions, including computation cost, materialization overhead, and data locality, making it impossible for any single static metric to achieve balanced execution across all threads.

Second, in skewed workloads, existing approaches suffer from significant cache contention, induced by dense substructures that are co-scheduled together. In SpGEMM, irregular sparsity forces practitioners to cache row-wise intermediates to reduce random access costs. However, dense substructures within skewed matrices incur excessive irregular random entry access, which creates heavy cache pressure and even induces significant cache contention, which has long been overlooked as a key performance factor. This cache contention is particularly pronounced under simultaneous multithreading (SMT), which aims to increase computation parallelism but worsens the interference.

In this paper, we address these challenges through a novel approach, *adaptive morsel scheduling* (AMS). In contrast to de facto methods, AMS revisits *dynamic scheduling*, a strategy conducive to load balancing that is often dismissed due to concerns regarding

<sup>4</sup>We use the following notation for R-MAT graphs [24]: ER, SSCA, and G500 denote non-skewed Erdős-Rényi, skewed SSCA, and highly skewed Graph500 graphs, respectively; *scale* sets the number of rows to  $2^{\text{scale}}$ , and *edgefactor* sets the average nonzeros per row.

scheduling overhead and poor performance [47, 84]. Through analysis (§ 2.2), we find that task granularity is the primary performance factor: a granularity that is too fine fails to preserve spatial locality and therefore suffers from excessive cache thrashing. Hence, we employ a morsel-wise dynamic scheduling strategy, where a “morsel”<sup>5</sup> is defined as a basic scheduling unit of several consecutive matrix rows. AMS ensures a morsel is large enough to maintain spatial locality, thereby greatly mitigating cache thrashing, and more importantly achieves load balance on skewed workloads through dynamic scheduling, effectively preventing computational idling and improving memory bandwidth utilization.

To alleviate cache contention, AMS introduces two lightweight and generalizable techniques: *contention-aware load balancing (CALB)*, which mitigates cache interference by adaptively scheduling dense morsels, and *data-conscious simultaneous multithreading activator (SMTact)*, which selectively enables SMT based on workload contention. These optimizations are simple in design, yet effective in practice, and easy to generalize to other sparse matrix kernels (e.g., SpMM and SpMV). Importantly, their performance benefits remain effective across heterogeneous bandwidth-boosted systems (e.g., CXL systems), ensuring that latency heterogeneity does not cause performance degradation.

Furthermore, the morsel-wise scheduling strategy enables a *hybrid accumulation mechanism (HYB)* for SpGEMM, where different accumulation algorithms can be selectively applied to morsels based on their intrinsic characteristics, thereby achieving more efficient execution and enhanced overall SpGEMM performance.

The contributions of this paper are summarized as follows:

- (1) We examine and characterize two co-existing root causes of performance degradation in skewed SpGEMM workloads: *multi-dimensional skew*, where no single static metric can balance all threads, and *cache contention*, where dense substructures induce severe on-chip interference that existing methods overlook.
- (2) We develop a novel morsel-wise dynamic scheduling framework, adaptive morsel scheduling (AMS), that systematically addresses both *multi-dimensional skew* and *cache contention* in SpGEMM. AMS decouples skew dimensions through fixed-result-size morsels, integrates two key techniques, contention-aware load balancing (CALB) and data-conscious simultaneous multithreading activator (SMTact), to alleviate on-chip cache interference, and enables hybrid accumulation mechanism (HYB) for morsel-adaptive accumulation. Our approach is straightforward in design, yet novel and generalizable, and effective in practice.
- (3) We evaluate AMS across a wide range of workloads, including matrix squaring, relational join-aggregate and join-project queries, graph analytics, and other sparse kernels, on both a main-memory and a CXL-augmented system, and show that it performs on par with state-of-the-art methods on non-skewed workloads while significantly outperforming them on skewed ones, addressing both computational idling and bandwidth underuse.

The rest of the paper is organized as follows. We first present the background and motivation in § 2, followed by a description

<sup>5</sup>We adopt the term from Leis et al. [72], where it originally denotes a fixed-size fragment of input data, whereas ours represents a group of consecutive rows with a roughly fixed output size, and we contrast the two paradigms in § 5.

---

#### Algorithm 1 Parallel Gustavson’s Algorithm for SpGEMM

---

```

1: Input: CSR matrices  $A \in \mathbb{R}^{m \times k}$  and  $B \in \mathbb{R}^{k \times n}$ 
2: Output: CSR matrix  $C \in \mathbb{R}^{m \times n}$ 
3: parallel for  $i \leftarrow 0$  to  $m - 1$  do            $\triangleright$  Parallel over rows of  $A$ 
4:   Initialize empty accumulator for row  $i$ 
5:   for all  $(k, A_{ik}) \in \text{row } i \text{ of } A$  do
6:     for all  $(j, B_{kj}) \in \text{row } k \text{ of } B$  do
7:       if  $j \in \text{accumulator}$  then
8:         accumulator[ $j$ ] +=  $A_{ik} \cdot B_{kj}$ 
9:       else
10:        accumulator[ $j$ ] ←  $A_{ik} \cdot B_{kj}$ 
11:      end if
12:    end for
13:  end for
14:  Store accumulator as row  $i$  of  $C$ 
15: end parallel for

```

---

of AMS in § 3. We then show the experimental results in § 4 and discuss related work in § 5. Finally, we conclude the paper in § 6.

## 2 BACKGROUND AND MOTIVATION

We first review state-of-the-art SpGEMM algorithms, then analyze how multi-dimensional skew and cache contention degrade performance under skewed workloads, and finally revisit dynamic scheduling as a promising solution, motivating our AMS approach.

### 2.1 The SpGEMM Kernel

SpGEMM computes the product  $C = AB$ , where  $A$ ,  $B$ , and  $C$  are all sparse matrices<sup>6</sup>. This form has proven effective across diverse large-scale workloads, including scientific computing, graph analytics, and more recently relational query processing [54, 83, 84, 102]. To reduce memory footprint at scale, sparse matrix formats are widely used, with compressed sparse row (CSR) being the most common representation in existing works [13, 22, 45, 61, 94]. However, the irregular and input-dependent sparsity patterns of sparse matrices, combined with irregular memory access patterns, make SpGEMM a fundamentally bandwidth-intensive workload, where performance depends critically on how effectively memory bandwidth is sustained across cores. Moreover, predicting the number and locations of nonzeros in the output matrix is often intricate, making memory pre-allocation challenging.

To address this pre-allocation challenge, SpGEMM is typically divided into two phases: a symbolic phase, which determines the nonzero structure of the output, and a numeric phase, which performs the actual computation based on that structure. In addition, the irregular nature of sparsity makes it difficult to efficiently gather and accumulate scattered contributions to each output entry. To tackle this, prior work has introduced a variety of accumulator designs to aggregate partial results during computation, including sorted variants such as heaps [83, 84], SPA [44, 50], and expand-sort-compress (ESC) [14, 31], as well as non-sorted variants based on hash tables [52, 83, 84].

<sup>6</sup>Throughout this paper, we refer to  $A$  and  $B$  in  $C = AB$  as the left-hand side and right-hand side matrices, respectively.

Building on these data representations, SpGEMM algorithms are commonly implemented using one of three formulations: inner-product [90], outer-product [2, 47], or row-row paradigm (Gustavson’s algorithm) [50]. Among these, Gustavson’s algorithm operates in a row-wise manner, allowing each row of the left-hand side matrix ( $A$ ) to be computed independently, enabling straightforward thread-level parallelization without the need for inter-thread synchronization (see Algorithm 1). It is this high degree of parallelism, combined with its efficient memory access patterns, that has made Gustavson’s algorithm a dominant formulation for SpGEMM across a wide range of computing platforms.

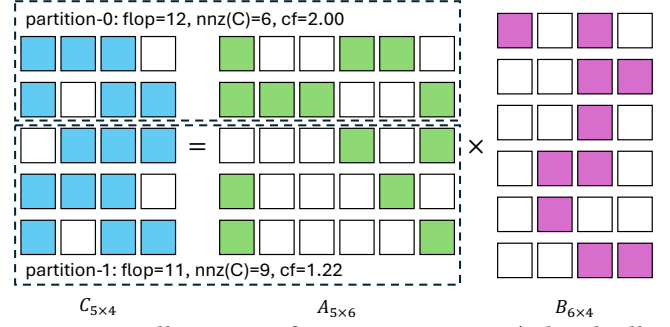
Following Gustavson’s algorithm, several load-balancing techniques have been proposed [2, 5, 11, 20, 26, 111], with static scheduling based on a flop-count scheme emerging as the dominant strategy [83, 84]. This method partitions the rows of matrix  $A$  into contiguous regions, each assigned approximately the same number of floating-point operations (i.e., additions plus multiplications). To achieve this, a lightweight preprocessing pass scans matrices  $A$  and  $B$  to estimate the flop count for each output row of  $C$  without performing actual multiplications. Based on these estimates, rows are grouped into partitions such that each thread receives a workload with a nearly equal number of floating-point operations. This approach enables static, flop-count-balanced scheduling across threads while avoiding runtime synchronization.

## 2.2 The Flop-Count-Based Scheme Under Skew

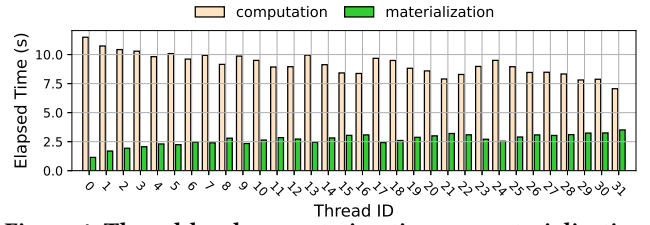
Despite its simplicity and widespread adoption [22, 84], the flop-count-based scheme is fundamentally limited and fails to achieve effective workload balance under skewed patterns. We identify two root causes, an inherent ignoring of multi-dimensional skew and the performance penalty of overlooking cache contention, both of which hurt load balancing and degrade bandwidth utilization.

(1) **Ignoring multi-dimensional skew.** Skewed SpGEMM workloads can exhibit imbalance across multiple dimensions, such as submatrix density and row-wise nonzero ( $nnz$ ) distribution. However, the flop-count-based scheme equalizes only the total number of floating-point operations, ignoring other forms of skew. As a result, one thread may receive a few dense rows (with high  $nnz$ ), while another is assigned many sparse rows. Although these assignments may have similar total flop counts, their structural differences lead to divergent runtime behaviors. Dense rows, in particular, induce “clustering”<sup>7</sup> effects during processing, which manifest as temporal bursts of memory and cache pressure. These effects significantly degrade performance, and the extent of impact varies by accumulation strategy [31, 44, 83]. Therefore, the scheme is inherently unable to preserve these skew dimensions.

Additionally, thread-level partitions often exhibit different compression factors ( $cf$ )<sup>8</sup>, which leads to divergent trends in computation and materialization times, making it challenging to effectively balance these two workload components. Consider Figure 3: using Gustavson’s algorithm, partition-0 (the upper two rows of  $A$ ) requires 12 flop counts<sup>9</sup>, producing 6 nonzeros; partition-1 (the



**Figure 3: An illustration of  $C = AB$  in SpGEMM (colored cells denote nonzero elements), where the two dashed boxes represent partitions with comparable flop counts but differing numbers of output nonzeros and compression factors.**



**Figure 4: Thread-level computation time vs. materialization time for hashing-based SpGEMM on a G500 skewed matrix (scale=19, edgfactor=8).**

bottom three rows) requires 11 flop counts but produces 9 nonzeros. Despite both involving comparable amounts of computation, the second has larger output size, inducing more materialization cost.

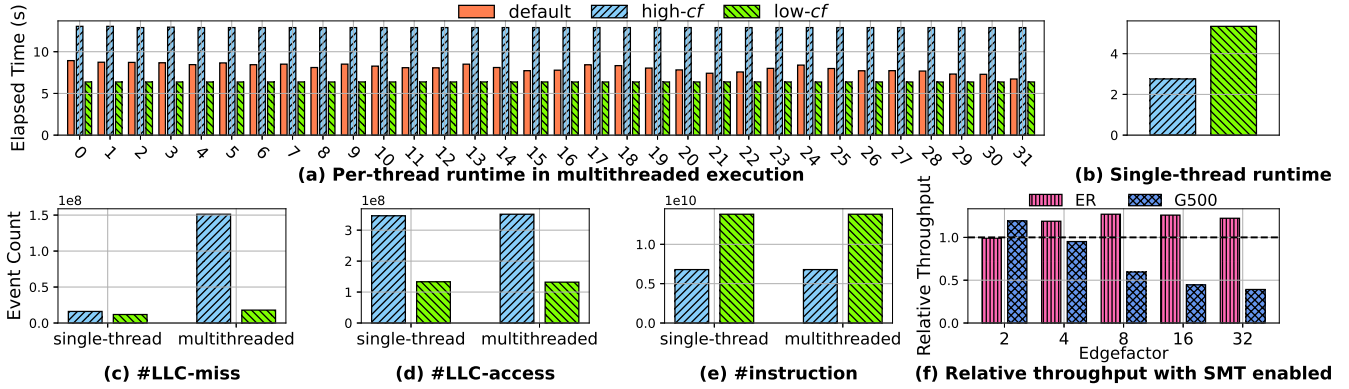
Empirical data on a skewed workload confirms this disparity, as Figure 4 shows: materialization time varies across threads, with computation time also exhibiting thread-level variation, but in the opposite trend. Moreover, these complexities are further compounded by hardware variability: performance sensitivity to the relative costs of computation versus memory access differs considerably across hardware platforms. This architectural variance makes it exceedingly difficult for purely flop-count-based schemes to establish a load balance that is both universally effective, and readily generalizable across diverse machines. Thus, such schemes are inherently limited in delivering robust load balancing.

(2) **Overlooking cache contention.** Existing methods also overlook the performance impact of cache contention during parallel execution. In Gustavson’s algorithm, each thread maintains an exclusive row-level accumulator whose size scales with the nonzero density of its assigned rows. Reading Figure 3 row-wise, each row of  $A$  gathers the  $B$  rows indexed by its nonzeros into that accumulator, so  $cf$  measures the nonzeros of  $B$  streamed per accumulator entry; a high- $cf$  row thus places significantly greater pressure on shared caches than a low- $cf$  row. As high- $cf$  partitions generally contain more high- $cf$  rows and Gustavson’s algorithm is typically parallelized row-wise, they are more prone to inter-thread cache interference than low- $cf$  partitions. Figure 5(a) shows this effect using the same skewed matrix as in Figure 4. When all threads compute their assigned default partitions, the runtime of a high- $cf$  partition

<sup>7</sup>We use the term “clustering” to describe this effect, following the notion of “primary clustering” in open-addressing hashing.

<sup>8</sup>The compression factor ( $cf$ ) is defined as the number of flop counts divided by the  $nnz$  of the result matrix  $C$ , i.e.,  $cf = \frac{flop}{nnz(C)}$  [47].

<sup>9</sup>Only multiplications are counted, as the number of additions is identical.



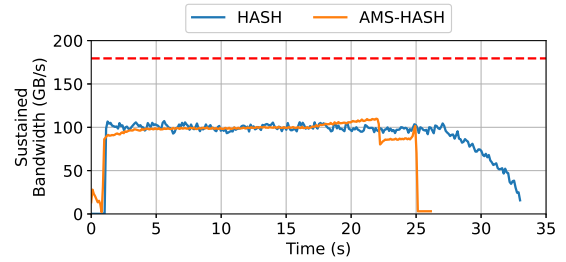
**Figure 5: (a) Per-thread computation time of HASH-based SpGEMM on a G500 skewed matrix (scale=19,  $edgefactor=8$ ) under three settings: all threads execute their default partitions; all execute a high- $cf$  partition (thread-0’s); all execute a low- $cf$  partition (thread-31’s). (b) Computation time of the low- $cf$  and high- $cf$  partitions under single-threaded execution. (c)–(e) LLC misses, LLC accesses, and instruction counts for the low- $cf$  and high- $cf$  partitions under both single-threaded and multithreaded execution. (f) Normalized throughput of HASH-based SpGEMM with SMT enabled on ER and G500 matrices (scale=18) across varying  $edgefactor$  values. The black dashed line marks the relative throughput without SMT.**

(originally assigned to thread-0 in the flop-count-based scheme) increases substantially: from 2.76 seconds in single-threaded execution (see Figure 5(b)) to 8.93 seconds under default multithreaded partitioning, and further to 13.01 seconds if all threads execute this high- $cf$  partition. Note that no sharing benefit arises in this case, as each thread operates on its own exclusive accumulator; instead, these accumulators compete for the same shared cache, amplifying interference as more threads process high- $cf$  regions. In contrast, a low- $cf$  partition (originally assigned to thread-31) sees only a modest runtime increase, from 5.36 to 6.73 seconds.

Further profiling reveals that cache contention is the root cause of this disparity: Figure 5(c)–(e) shows a sharp rise in last-level cache (LLC) misses for the high- $cf$  region, while instruction count and LLC accesses remain relatively stable, confirming cache contention as the primary bottleneck<sup>10</sup>.

The problem is further exacerbated under simultaneous multi-threading (SMT): Figure 5(f) shows that non-skewed workloads (e.g., ER matrices with typically low  $cf$ ) benefit from SMT, whereas skewed workloads (e.g., G500 matrices with high- $cf$  subregions) suffer increasing throughput degradation as density grows, since higher density amplifies the skew and thus intensifies SMT-induced contention. This underscores the importance of contention-aware scheduling; neglecting this factor not only compromises load balance but also degrades overall SpGEMM efficiency.

Ignoring multi-dimensional skew and overlooking cache contention together compound into a macro-level penalty on system-wide bandwidth utilization. As Figure 6 illustrates on a skewed workload, bandwidth resources are only fully utilized during the initial burst of parallel execution; as straggler cores emerge, system-wide bandwidth consumption starts to drop, leaving expensive memory resources idle. This renders prevailing bandwidth expansion technologies, such as CXL, far less effective, as the slowest thread governs total runtime, and its prolonged latency [105] further widens the gap despite the increased aggregate bandwidth



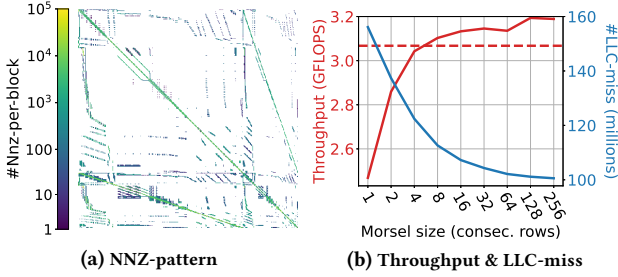
**Figure 6: Comparison of sustained bandwidth of the numeric phase between flop-count-based hashing-based SpGEMM and the AMS variant on a G500 skewed workload (scale=19,  $edgefactor=16$ ); the horizontal red dashed line denotes system memory bandwidth (179.50 GB/s).**

available. In contrast, a contention-aware load-balancing approach (our AMS method) sustains markedly higher bandwidth execution.

### 2.3 Revisiting Dynamic Scheduling

Having established that both multi-dimensional skew and cache contention undermine existing scheduling strategies, we now revisit the scheduling paradigm that enables our solution. Specifically, we revisit *dynamic scheduling* [84, 88]: rows of  $A$  are placed into a shared task pool, and threads repeatedly fetch and process one row at a time, achieving nearly 100% load balance. While simple and theoretically well-balanced, this approach has been shown to underperform compared to flop-count-based scheduling and has largely been dismissed in recent literature, which attributes the performance penalty primarily to the synchronization overhead associated with dynamic task dispatch [47, 84]. However, we contend that it is task granularity, rather than synchronization overhead, that plays the dominant role in determining performance. When task size is carefully managed, dynamic scheduling can match, if not outperform, flop-count-based schemes.

<sup>10</sup>Write-out time is excluded to directly address computation-level contention.



**Figure 7: (a) Nonzero pattern of the `vas_stokes_1M` matrix. (b) Throughput and LLC misses of hashing-based SpGEMM on `vas_stokes_1M` using dynamic scheduling with varying morsel sizes (measured in consecutive rows per morsel); the horizontal red dashed line indicates the throughput of the flop-count-based scheme.**

Take a real-world SpGEMM workload, `vas_stokes_1M` [32, 99], for example. The matrix exhibits clear spatial locality across rows, particularly within blocks of consecutive rows (Figure 7(a)). We benchmark the performance of hashing-based SpGEMM using dynamic scheduling with varying task granularities (i.e., the number of consecutive rows per task). Figure 7(b) shows that throughput improves with increasing granularity, plateauing around 128 rows, and surpasses its flop-count-based counterpart due to better load balance. Further analysis of LLC miss counts reveals that larger task sizes result in fewer misses, suggesting better preservation of spatial locality. It is worth noting that many real-world matrices exhibit strong spatial locality in their nonzero patterns, and prior works have introduced several offline reordering algorithms to enhance locality for downstream operations [5, 26, 111]. Based on these observations, we maintain that dynamic scheduling remains a practical and efficient strategy as long as spatial locality is effectively preserved. We therefore build on this insight for the design of our AMS approach.

### 3 METHODOLOGY

This section presents our proposed framework, adaptive morsel scheduling (AMS). We begin with the core concept of morsel-wise dynamic scheduling, our strategy for addressing multi-dimensional skew and achieving efficient load balancing in SpGEMM. Building upon this foundation, we present two key techniques, contention-aware load balancing (CALB) and data-conscious simultaneous multithreading activator (SMTact), designed specifically to alleviate on-chip cache contention. Furthermore, we explain how the morsel-wise framework enables our hybrid accumulation mechanism (HYB) approach for optimizing SpGEMM execution. Finally, we discuss three broader implications of AMS, namely the interplay between load balance and cache efficiency that governs the benefit of bandwidth-expanded memory, the carry-over of the underlying design principle to relational and graph processing, and the extension of AMS to other sparse matrix multiplication kernels.

We note that AMS targets the numeric phase of SpGEMM, which dominates both computation and memory traffic. While the same ideas can be applied to the symbolic phase, the benefits are less pronounced; for simplicity, we use standard dynamic scheduling

with morsels of a fixed row count for that phase. We defer a detailed discussion of the symbolic phase, together with a breakdown of its cost, to Appendix A.

Before detailing our techniques, we briefly digress to outline three key design principles that an efficient SpGEMM solution should satisfy, all of which are met by our proposed AMS.

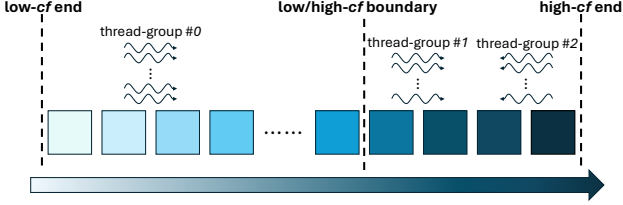
- (1) **Simple and lightweight.** SpGEMM has been extensively optimized over decades [40], and any added complexity can degrade its performance. This is especially critical when SpGEMM is repeatedly invoked as a core kernel in real-world systems [22, 48, 61, 76, 84, 112]. Thus, any proposed method must be simple and lightweight to avoid introducing significant execution penalties.
- (2) **Non-disruptive to non-skewed workloads.** While skewed patterns are common in real-world workloads, non-skewed distributions (e.g., uniform distribution, symmetric distribution) also arise frequently. A design that benefits skewed cases but penalizes non-skewed ones forces practitioners to make case-dependent trade-offs, increasing system complexity and operational overhead.
- (3) **Generalizable across algorithms and platforms.** An effective SpGEMM solution should generalize across algorithmic variants and hardware platforms. In practice, SpGEMM is used in diverse contexts with varying requirements, e.g., numerical computing often demands sorted output [31], while relational and graph queries typically accept non-sorted results [5, 26, 45]. Also, it should adapt to different hardware architectures, where sensitivities to compute and memory bottlenecks vary. A practical solution must therefore accommodate both, supporting sorted and non-sorted pipelines while maintaining robust performance across platforms.

#### 3.1 Morsel-Wise Dynamic Scheduling

We previously demonstrated that, by using morsels composed of a sufficient number of consecutive rows, dynamic scheduling is able to effectively preserve spatial locality in sparse matrices. This method significantly reduces LLC misses with negligible synchronization overhead (§ 2.3). We therefore adopt this line of design for SpGEMM execution.

While scheduling work in morsels of a fixed row count helps reduce cache thrashing, we argue that this approach leaves room for further improvement, as it does not address the *multi-dimensional skew* identified in § 2.2. Specifically, it has two limitations: (1) Fixed-row-count morsels do not account for differences in flop count or result *nnz*, making it difficult to balance computation and materialization costs. This limitation is particularly pronounced in skewed workloads, where high-*cf* submatrices incur high computational expense while others involve substantial write-out overhead. Due to such disparities in two dimensions, modeling inter-thread cache contention becomes challenging. (2) This fixed-row strategy also fails to control the size of the resulting output. A single morsel containing a few dense rows, particularly in skewed workloads, can produce more data than the CPU cache can accommodate, making it challenging to construct a cache-aware analysis model [52, 82, 96].

To address these issues, we propose scheduling morsels based on a fixed result size, i.e., a target  $nnz(C)$  per morsel. Recall that SpGEMM consists of two phases, where the symbolic phase determines the per-row nonzero structure. We thus leverage this information to form morsels of consecutive rows in an adaptive



**Figure 8: Thread scheduling with contention-aware load balancing (CALB).** Darker colors indicate higher compression factors ( $cf$ ).

manner, such that each morsel contains approximately the same number of output nonzeros. By fixing the materialization cost per morsel, this approach normalizes one dimension of skew, allowing us to focus on the remaining computational cost dimension. It also decouples  $cf$  from the output size that flop-count-based partitioning leaves uncontrolled: with  $nnz(C)$  fixed,  $cf$  varies solely with the flop count.

Additionally, to facilitate on-chip cache modeling, we limit the morsel size to half of the L2 cache capacity <sup>11</sup>.

We stress that this size is a *soft target* rather than a hard bound, and that it budgets only the output footprint  $nnz(C)$  of a morsel, not the accumulator or the rows of  $B$  streamed into it. AMS appends whole consecutive rows to a morsel until the target is reached, and because an output row is atomic in the row-parallel Gustavson formulation, AMS never splits a row across threads. A row whose own output already exceeds the target therefore forms a morsel by itself and may be far larger than the target. Such oversized rows are common rather than exceptional in skewed workloads, and we show in Appendix B that they do not degrade AMS.

This strategy is simple and efficient, and it forms the foundation for our subsequent designs. More importantly, it introduces no additional algorithmic phase, fully aligning with our design principle of being **simple and lightweight**.

### 3.2 Contention-Aware Load Balancing

We now elaborate on our alleviation of inter-thread on-chip cache contention, which can degrade SpGEMM performance even under balanced workload distribution. Since we form morsels based on a fixed result size, we can easily compute the compression factor ( $cf$ ) for each morsel. Recall from § 2.2 that contention originates at the row level: a high- $cf$  row streams more nonzeros of  $B$  per accumulator entry (see Figure 3), and is thus contention-prone regardless of how rows are grouped. With  $nnz(C)$  fixed per morsel, a high- $cf$  morsel likewise carries more computation against an identical footprint, and remains more prone to contention than a low- $cf$  one. Therefore, we aim to co-execute high- $cf$  and low- $cf$  morsels concurrently, alleviating cache contention and reducing susceptibility to inter-thread interference.

To that end, we first sort all morsels in ascending order of  $cf$  and empirically set a boundary between low- $cf$  and high- $cf$  regions, typically between the 80th and 95th percentile. This is motivated by the observation that most skewed workloads follow a power-law

distribution [5, 26, 111, 127], where a small fraction of dense rows contribute to the majority of the workload. We then divide the available threads into three groups: thread group #0 starts from the low- $cf$  end and progresses toward the high- $cf$  end; group #1 begins at the low/high- $cf$  boundary and proceeds toward the high- $cf$  end; group #2 starts at the high- $cf$  end and moves backward toward the boundary (see Figure 8). The majority of threads are allocated to group #0 to handle low- $cf$  tasks, while smaller subsets are assigned to groups #1 and #2 for high- $cf$  regions. This arrangement ensures that only a limited number of high- $cf$  morsels are processed concurrently. Dividing the remaining threads into groups #1 and #2 further helps mitigate cache contention by avoiding simultaneous execution of multiple contention-prone morsels at the high- $cf$  end. Meanwhile, co-running low- $cf$  morsels, which are less sensitive to cache interference, alongside high- $cf$  morsels helps absorb contention effects and improves overall execution efficiency.

While this scheme alleviates cache contention, it does not guarantee load balance. Some thread groups may finish early and become idle, leading to underutilization of computational and memory bandwidth resources. To address this, we allow early-finished threads to assist other active groups. Specifically, if group #0 reaches the boundary early, it merges with group #1 to continue processing toward the high- $cf$  end. Likewise, early-finished threads from groups #1 and #2 join group #0 to help complete tasks in the low- $cf$  region. This cooperative mechanism keeps all threads active, ensuring efficient use of computation and memory bandwidth.

The CALB scheme is conceptually simple, yet novel and highly effective. To the best of our knowledge, it is the first SpGEMM load-balancing technique that explicitly considers on-chip cache contention as a scheduling factor and thereby delivers tangible performance benefits. Moreover, the method is platform-agnostic and can be seamlessly integrated into any SpGEMM algorithm based on Gustavson’s formulation, adhering to the design principle of being **generalizable across algorithms and platforms**.

### 3.3 Data-Conscious Simultaneous Multithreading Activator

While SpGEMM is generally considered bandwidth-bound, its computational phase still incurs a non-negligible overhead. This overhead can typically be mitigated by leveraging simultaneous multithreading (SMT), which increases the logical core count to effectively improve computational throughput. However, as shown in Figure 5(f), this performance benefit from SMT is primarily observed in non-skewed workloads; in contrast, skewed workloads degrade significantly when SMT is enabled.

We attribute this disparity to the increased on-chip cache pressure from SMT. Although SMT increases the logical thread count, it does not expand the physical cache. Consequently, high- $cf$  morsels suffer from intensified cache contention, an issue that is particularly exacerbated in skewed workloads where  $cf$  values are disproportionately large for a few morsels. These specific morsels are highly vulnerable to SMT-induced interference, which in turn leads to substantial performance degradation.

<sup>11</sup>We empirically find that a morsel size ranging from  $\frac{1}{8}$  to  $\frac{1}{2}$  of the L2 cache offers the best trade-off between performance and cache contention.

To mitigate this issue while preserving the computational benefits of SMT for non-skewed workloads, we adopt a selective activation strategy of SMT, namely data-conscious simultaneous multi-threading activator (SMTact). Specifically, we set a threshold based on the highest  $cf$  observed. SMT is disabled if the workload contains any morsels exceeding this threshold, and enabled otherwise. This strategy ensures that SMT is applied only when the workload is less prone to cache contention<sup>12</sup>. The approach is simple yet effective, as it balances the computational benefits of SMT against the risk of elevated cache contention, aligning with our second design principle of being **non-disruptive to non-skewed workloads**.

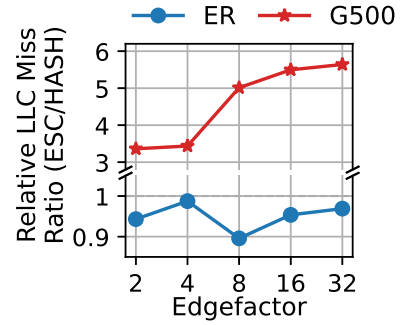
### 3.4 Hybrid Accumulation Mechanism

Due to the inherently sparse and irregular nature of SpGEMM, computing a single output row generates multiple intermediate sparse products. These products must then be merged, a process that requires appropriate accumulators to mitigate two challenges: severe irregular memory access patterns and the high cost of managing collisions for identical column indices. Recognizing these challenges, numerous works have focused on developing efficient accumulators [31, 44, 84]. However, the effectiveness of any given accumulation strategy varies dramatically based on the characteristics of the underlying matrix morsel. This makes a static, one-size-fits-all accumulator inherently suboptimal.

To address this, we leverage the morsel-wise framework of AMS to introduce a hybrid accumulation mechanism (HYB) that dynamically selects the most suitable accumulator for each morsel. Specifically, we make this choice based on the aforementioned morsel-wise  $cf$  value, selectively choosing between two widely used accumulators: hash tables (HASH) [83, 84], which use linear probing and multiplicative hashing to reduce collisions, and expand-sort-compress (ESC) [31, 47], which relies on highly optimized radix sorting to alleviate common logarithmic sorting overheads.

The choice between these two accumulators is highly dependent on the morsel-wise  $cf$ . As ESC generates sorted output, it executes more instructions than HASH. However, this instruction overhead is not necessarily detrimental, particularly for low- $cf$  morsels. For such morsels, the number of intermediate products is similar to the final number of nonzeros, leading to a moderate cost for merging and sorting. In contrast, HASH becomes more populated for low- $cf$  morsels, and the overhead of hash computation and probing becomes more pronounced. For high- $cf$  morsels, however, HASH is the better choice, as a massive number of intermediates are hashed into a few slots, greatly reducing potential cache misses. Conversely, ESC must pay a significant cache miss cost to merge a large volume of intermediate results into a small final output. Figure 9 corroborates this by microbenchmarking the LLC misses of HASH and ESC, which shows that ESC suffers significantly as the  $cf$  increases (e.g., in skewed G500 workloads) compared to non-skewed (ER) workloads.

Therefore, we empirically identify a  $cf$  threshold, using ESC for morsels with a  $cf$  below this value and opting for HASH for morsels above it<sup>13</sup>. This morsel-wise selection also offers an architectural



**Figure 9: The LLC miss ratio of ESC normalized to HASH (i.e., LLC Misses of ESC / LLC Misses of HASH), evaluated on ER and G500 datasets at Scale 18 with varying edgefactors (higher edgefactors suggest denser workloads).**

benefit: selecting the accumulator on a per-morsel basis alleviates the instruction-level branch-misprediction costs that finer-grained (per-row) selection would incur.

It is worth noting that by including HASH, our HYB approach may produce a non-sorted result matrix, which is acceptable for applications like relational queries and graph analytics that do not require sorted output [59, 83]. However, if sorted output is necessary, HYB can be easily adapted: instead of choosing between HASH and ESC, it can be configured to choose the efficient strategy from a set of sorted accumulators, such as ESC, HEAP [84], SPA [44], or even a hash-then-sort approach [28], to ensure a fully sorted output. Consequently, this adaptive approach ensures each morsel is processed with the most suitable accumulator, leading to better overall SpGEMM performance regardless of the output ordering requirement.

### 3.5 Implications and Broader Applicability

Having presented AMS’s components, we now step back to consider what they imply beyond the SpGEMM kernel itself. AMS highlights an important interaction between load balance and cache efficiency. Static partitioning preserves locality but leaves cores idle under skew, thereby underutilizing memory bandwidth; dynamic scheduling keeps cores busy but raises cache contention once the working sets of co-executing morsels exceed the shared cache. Effective scheduling must therefore maintain enough concurrency to saturate the memory system while limiting avoidable cache traffic, which is what AMS achieves by scheduling morsels from cheap metadata. This interaction is precisely what makes bandwidth-expanded memory [117] pay off, since additional bandwidth is realized only when enough cores issue concurrent requests, and it is why combining balanced execution with reduced interference lets AMS sustain throughput beyond the local-DRAM peak (§ 4.4).

The same design principle can apply beyond SpGEMM when lightweight metadata is available to estimate each morsel’s processing cost and demand on the shared cache. For SpGEMM, the symbolic phase provides this information directly. Other operators can obtain approximate metadata through a preliminary statistics pass [9, 27] or sampling [114, 116]; exact estimates are unnecessary

<sup>12</sup>We also experiment with using quantiles of the  $cf$  distribution instead of the maximum, and observe comparable performance with an appropriately chosen threshold.

<sup>13</sup>We find that a  $cf$  of 1.5–1.7 provides a robust threshold for selecting the appropriate accumulator.

because CALB uses only the relative ordering of morsels by cache demand and SMTact only tests whether a threshold is crossed. This creates opportunities to apply contention-aware morsel scheduling to other operators over relational and graph data with skewed, cache-resident state [3, 33, 45, 59, 104]. For example, a morsel-driven engine [35, 72] could use partition-size estimates to approximate cache demand when scheduling hash-aggregation or hash-join tasks over skewed keys.

A direct and notable extension of AMS, beyond SpGEMM and the relational query and graph operators it underpins, is to apply it atop other sparse matrix multiplication kernels, e.g., sparse matrix-dense matrix multiplication (SpMM) and sparse matrix-vector multiplication (SpMV), which multiply a sparse matrix by a dense operand. Their output is dense and of fixed size, which renders HYB unnecessary and makes morsel formation by a target result size meaningless; we instead group consecutive rows by a target *nnz* of the left-hand operand. Although the skew thereby reduces to a single dimension (flop count), load imbalance and cache contention persist, as morsels of many short rows and morsels of few long rows differ markedly in data reuse [41, 121], so CALB and SMTact apply as they do in SpGEMM [75]. Since most SpMM and SpMV solutions likewise follow a Gustavson-style row-wise formulation, the adaptation is straightforward, and we report the resulting speedups in § 4.5.

## 4 EXPERIMENTAL EVALUATION

This section evaluates AMS in addressing multi-dimensional skew and on-chip cache contention. We begin with the experimental setup (§ 4.1), then benchmark AMS on matrix squaring across diverse workloads (§ 4.2) and dissect CALB and SMTact to validate their contention alleviation (§ 4.3). We further evaluate AMS on a bandwidth-expanded CXL system to show improved bandwidth utilization (§ 4.4). We then demonstrate its broad applicability on other sparse matrix multiplication kernels (§ 4.5), on relational join-aggregate and join-project queries (§ 4.6), and on graph-analytic workloads (§ 4.7), including PathSim meta-path construction [104], breadth-first search, triangle counting, and an end-to-end Markov clustering pipeline (HipMCL). We close with a discussion (§ 4.8) that distills the key findings across these workloads and platforms, and relates them back to the design rationale of AMS.

### 4.1 Experimental Setup

We conduct experiments on a single-socket machine running Linux kernel 6.11.0. This system is equipped with an Intel Xeon® Gold 6430 CPU, which features 32 physical cores and supports 64 logical threads via simultaneous multithreading (SMT)<sup>14</sup>. Each core includes a 48 KB L1 data cache, a 32 KB L1 instruction cache, and a 2 MB L2 cache, while all cores share a 60 MB last-level cache (LLC). The system comprises 192 GB of DDR5 DRAM, providing 179.50 GB/s of memory bandwidth. We also conduct experiments on a CXL-augmented memory system to validate the generalizability of AMS, with the setup details deferred to § 4.4.

<sup>14</sup>We disable Intel Speed Select Technology - Base Frequency to ensure equal performance for all physical cores [110].

**Table 1: Summary of 20 representative matrices evaluated.**

| Matrix         | row(A)  | nnz(A)  | gini(A) <sup>†</sup> | flop <sup>‡</sup> | nnz(A <sup>2</sup> ) | cf     |
|----------------|---------|---------|----------------------|-------------------|----------------------|--------|
| hugetric-00020 | 7.13M   | 21.37M  | 0.0003               | 64.08M            | 44.61M               | 1.4363 |
| GL7d18         | 1.96M   | 35.60M  | 0.0395               | 890.29M           | 843.47M              | 1.0555 |
| 333SP          | 3.72M   | 22.22M  | 0.0559               | 134.79M           | 71.97M               | 1.8730 |
| NLR            | 4.17M   | 24.98M  | 0.0736               | 152.77M           | 82.02M               | 1.8626 |
| M6             | 3.51M   | 21.01M  | 0.0742               | 128.49M           | 68.99M               | 1.8625 |
| AS365          | 3.80M   | 22.74M  | 0.0765               | 138.91M           | 74.62M               | 1.8616 |
| auto           | 448.70K | 6.63M   | 0.1016               | 101.25M           | 33.06M               | 3.0634 |
| delaunay_n23   | 8.39M   | 50.34M  | 0.1220               | 316.97M           | 173.67M              | 1.8251 |
| nv2            | 1.46M   | 37.48M  | 0.1401               | 1.28B             | 292.14M              | 4.3475 |
| Freyscale1     | 3.43M   | 17.06M  | 0.1858               | 90.33M            | 38.31M               | 2.3580 |
| gsm_106857     | 589.45K | 21.76M  | 0.2240               | 947.26M           | 132.40M              | 7.1546 |
| amazon-2008    | 735.33K | 10.32M  | 0.2856               | 218.17M           | 63.16M               | 3.4546 |
| webbase-1M     | 1.01M   | 3.11M   | 0.3455               | 1.37B             | 1.31B                | 1.0475 |
| vas_stokes_2M  | 2.15M   | 65.13M  | 0.3771               | 6.95B             | 1.17B                | 5.9452 |
| rel9           | 9.89M   | 23.67M  | 0.4016               | 2.10B             | 2.04B                | 1.0300 |
| vas_stokes_1M  | 1.10M   | 34.77M  | 0.4081               | 4.39B             | 721.63M              | 6.0776 |
| patents        | 3.78M   | 14.98M  | 0.6585               | 178.30M           | 132.12M              | 1.3496 |
| cit-Patents    | 3.78M   | 16.52M  | 0.6844               | 239.46M           | 162.35M              | 1.4749 |
| NotreDame_www  | 325.73K | 929.85K | 0.7825               | 418.01M           | 288.98M              | 1.4465 |
| web-NotreDame  | 325.73K | 1.50M   | 0.8523               | 503.57M           | 293.20M              | 1.7175 |

<sup>†</sup> Gini coefficient [69] of the row-wise nonzeros in A.

<sup>‡</sup> Only multiplications are counted, as additions are equal in number.

Following prior settings [6, 20, 22, 23, 83], we conduct our primary performance analysis using R-MAT matrix squaring workloads [24, 42, 64]. We employ three representative sets of R-MAT matrices to model different workload characteristics: ER ( $a = b = c = d = 0.25$ ), SSCA ( $a = 0.60, b = c = 0.13, d = 0.14$ ), and G500 ( $a = 0.57, b = c = 0.19, d = 0.05$ ), which represent non-skewed, skewed, and highly skewed workloads, respectively. All R-MAT generators support control over size and density via two parameters: *scale* (determining the number of rows, i.e.,  $2^{scale}$ ) and *edgfactor* (defining the average nonzeros per row). We sweep *scale* over 19–21 for ER, 18–20 for SSCA, and 17–19 for G500, with *edgfactor* over 2–32 in all three. For G500 workloads, higher *edgfactor*s amplify multi-dimensional skew and cache contention (§ 2.2). Additionally, we evaluate SpGEMM performance using real-world workloads from the SuiteSparse Matrix Benchmark [32] to validate the robustness of AMS. We perform matrix squaring on 119 SuiteSparse matrices, with row counts ranging from 100K to 10M and nonzero counts from 146K to 316M. This range is deliberate, since even the smallest matrix in it does not fit in the 2 MB private L2 cache<sup>15</sup>, so that every workload genuinely exercises the cache hierarchy under study rather than residing in a private cache, while the largest ones still complete on our platform without triggering out-of-memory errors. For a clear presentation, we present results for 20 representative matrices (Table 1), ordered by their Gini coefficient<sup>16</sup>, as other matrices show similar trends. Beyond SpGEMM, we evaluate AMS on relational join-aggregate and join-project queries, on graph-analytic workloads such as PathSim meta-path construction [104], and on other sparse matrix multiplication kernels, with setups and workloads detailed in § 4.6, § 4.7, and § 4.5, respectively.

<sup>15</sup>Storing a matrix of  $n$  rows and  $nnz$  nonzeros in CSR with 8-byte indices and double-precision values takes  $8(n + 1) + 16 nnz$  bytes, so the lower end of this range already occupies about 3 MB, whereas the upper end occupies about 5.1 GB.

<sup>16</sup>The Gini coefficient measures inequality in the row-wise nonzero distribution; a value of 0 indicates uniform distribution, while values approaching 1 indicate extreme skew. Coefficients above 0.30 generally indicate skewed workloads [32, 69].

**Table 2: Summary of all evaluated SpGEMM methods**

| Method                                                       | Category   | Sorted | Description                                                                    |
|--------------------------------------------------------------|------------|--------|--------------------------------------------------------------------------------|
| <i>Algorithmic baselines (built upon by our AMS methods)</i> |            |        |                                                                                |
| HASH [84]                                                    | Baseline   | ✗      | Hash table accumulator with flop-count-based scheduler.                        |
| ESC [31]                                                     | Baseline   | ✓      | Radix-sort (expand-sort-compress) accumulator with flop-count-based scheduler. |
| <i>External / industry competitors</i>                       |            |        |                                                                                |
| HEAP [84]                                                    | Competitor | ✓      | Heap accumulator with flop-count-based scheduler.                              |
| PB [47]                                                      | Competitor | ✓      | Outer-product SpGEMM flow with flop-count-based scheduler.                     |
| MKL [61]                                                     | Competitor | ✗      | Intel oneMKL SpGEMM — non-sorted variant.                                      |
| MKLS [61]                                                    | Competitor | ✓      | Intel oneMKL SpGEMM — sorted variant.                                          |
| <i>Proposed AMS-based methods</i>                            |            |        |                                                                                |
| AMS-HASH                                                     | Proposed   | ✗      | AMS approach using a HASH accumulator.                                         |
| AMS-ESC                                                      | Proposed   | ✓      | AMS approach using an ESC accumulator.                                         |
| AMS-HYB                                                      | Proposed   | ✗      | AMS with per-morsel adaptive HASH/ESC selection.                               |

We integrate our AMS framework into two prevalent Gustavson-based SpGEMM algorithms: the non-sorted HASH algorithm [83, 84] and the sorted ESC algorithm [14, 31]. These are referred to as AMS-HASH and AMS-ESC, respectively<sup>17</sup>. Moreover, we implement our hybrid accumulation mechanism (HYB) as AMS-HYB, which adaptively selects between HASH and ESC accumulators. We empirically tune the parameters for AMS-based approaches, configuring morsel size at half the L2 cache size, and distributing threads using a 60%-25%-15% ratio across groups #0, #1, and #2, respectively, for optimal CALB performance.

We compare our methods against several state-of-the-art baselines (see Table 2 for an overview). These include: (1) the same HASH and ESC algorithms driven by a state-of-the-art flop-count-based scheduler; (2) the sorted HEAP algorithm [83, 84], which exploits the implicit column ordering of input matrices to reduce sorting overhead at the cost of increased memory usage; (3) the outer-product-based PB method [47]; and (4) the de facto industry solution, Intel MKL [61], including both its sorted (MKLS) and non-sorted (MKL) variants. While other SpGEMM algorithms, such as SPA [44] and SaSpGEMM [52], were also evaluated, we omit their results for conciseness as they consistently underperformed.

All implementations are compiled using GCC-11.5.0 with the -O3 optimization flag and parallelized with OpenMP [30]. Baselines are manually tuned for optimal performance, which includes enabling SMT for non-skewed workloads and disabling it for skewed ones. Our AMS-based methods are tuned with equal care, sweeping the thread group sizes, the low/high- $cf$  boundary, and the SMTact threshold. As in prior works, throughput, measured in GFLOPS (giga floating-point operations per second), serves as the primary evaluation metric. Unless a measurement is explicitly labeled as numeric-phase profiling, all reported runtimes and throughputs cover the complete SpGEMM pipeline, i.e., symbolic execution, prefix sum and output allocation, AMS preparation, numeric execution, and output finalization. All reported results are averaged over 10 runs<sup>18</sup>.

<sup>17</sup>Sorted algorithms generally incur more overhead to produce row-wise sorted output.

<sup>18</sup>The source code, data, and/or other artifacts have been made available at <https://github.com/fukien/casmm>.

## 4.2 Matrix Squaring Performance Evaluation

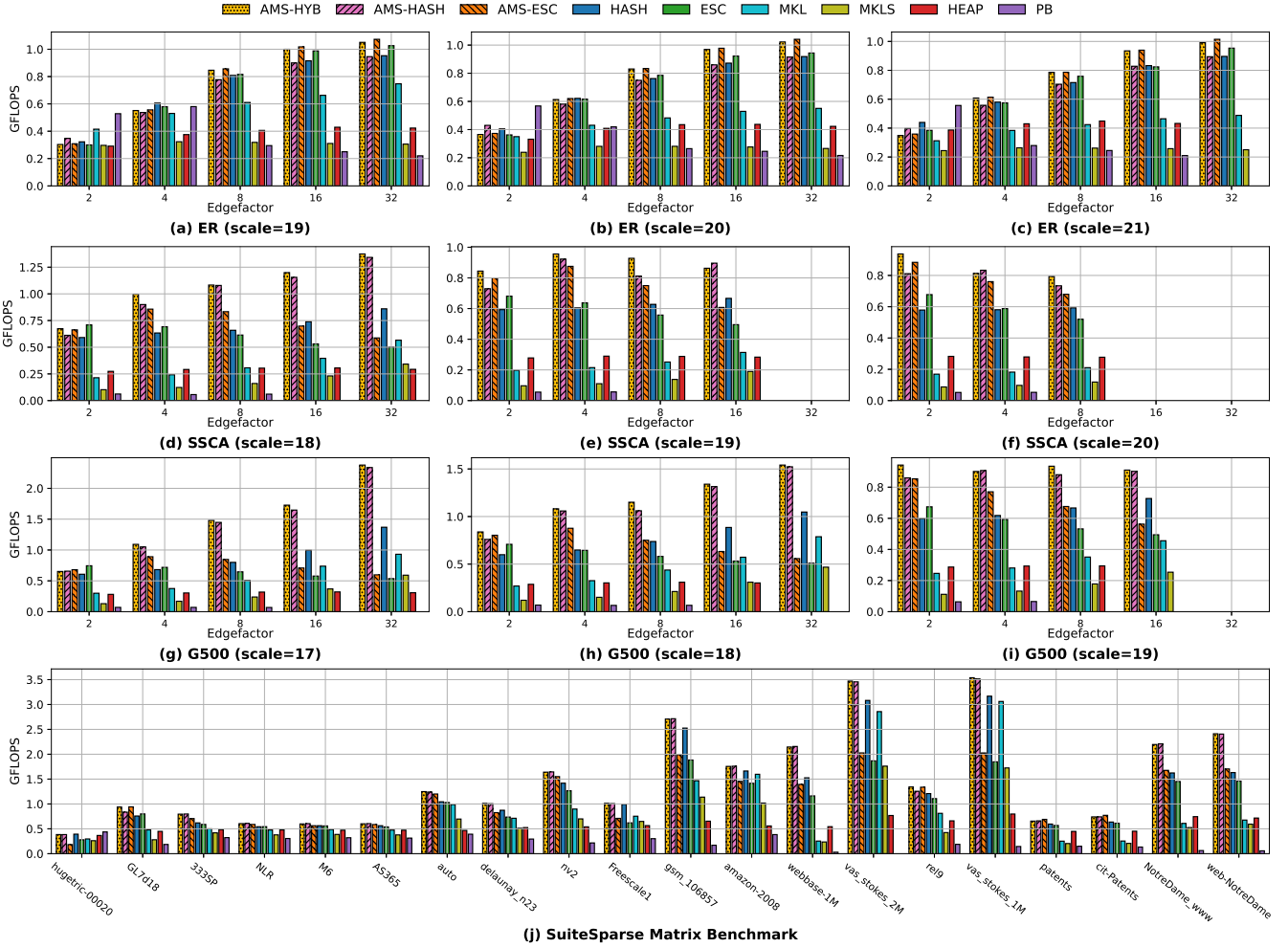
Figure 10 presents the matrix squaring performance on R-MAT and SuiteSparse workloads, and shows that our AMS-based algorithms consistently achieve high performance across this broad spectrum. Specifically, the proposed AMS-HYB is generally the superior solution: it maintains performance comparable to AMS-HASH in skewed workloads (SSCA and G500) while remaining as competitive as AMS-ESC in non-skewed, high-*edgefactor* workloads (ER). This outcome validates both the effectiveness of our HYB design and the soundness of our  $cf$ -based criterion for adaptively selecting the best morsel-wise accumulator for efficient multiplication.

Beyond the superior performance of AMS-HYB, the AMS framework itself significantly boosts both its non-sorted (AMS-HASH) and sorted (AMS-ESC) variants. These methods perform comparably to their respective baselines on ER matrices but deliver a substantial performance advantage on the skewed SSCA and G500 matrices. The advantage moreover tracks the degree of skew for the non-sorted variant: at matched *scale* and *edgefactor*, where SSCA and G500 differ only in their R-MAT probabilities, AMS-HASH gains more on the more skewed G500 side in both such settings, reaching 1.44× against 1.23× at *scale* 19 and *edgefactor* 2 (Appendix B). This demonstrates that AMS’s dynamic scheduling, when paired with appropriately sized morsels, effectively addresses multi-dimensional skew on skewed workloads without sacrificing performance on non-skewed ones. The strong results on ER matrices (Figure 10(a)–(c)) also validate the effectiveness of SMTact, which selectively enables SMT in low-contention scenarios to enhance computational throughput, a critical factor in non-skewed workloads.

Comparing the impact of AMS across different algorithms, we observe that the performance improvement is more pronounced in HASH than in ESC. This is because ESC relies on radix sort, which has a complexity of  $O(\omega \cdot n)$  and correlates directly with the flop count. Although HASH-based methods also exhibit  $O(n)$  complexity in hash table lookups, their efficiency is more susceptible to “clustering” effects, making flop-count-based models less effective in accurately estimating cost and balancing workloads. In contrast, our AMS approach is agnostic to the underlying SpGEMM algorithm. For both non-sorted and sorted algorithms, AMS achieves performance gains of up to 81.01% and 36.03%, respectively, demonstrating its general applicability across algorithmic variants.

To further illustrate the source of these gains, Figure 11 drills down into the per-thread elapsed time, comparing AMS-based methods against their respective baselines on a representative skewed workload. The baseline HASH and ESC exhibit significant thread-level runtime variance, confirming that flop-count-based scheduling fails to balance the actual workload under skew. In contrast, AMS-based variants achieve substantially more uniform per-thread runtimes, demonstrating that our scheduling framework effectively addresses the load imbalance problem inherent in skewed workloads.

Despite these advantages, Figure 10 shows that AMS-based algorithms may occasionally underperform their flop-count-based counterparts in highly sparse cases (e.g., G500 with *scale* 17 and *edgefactor* 2), primarily due to overhead from  $cf$ -based morsel



**Figure 10: SpGEMM performance evaluation.** Subfigures (a)–(i) present results for R-MAT matrices (ER, SSQA, G500) across varying scales and *edgfactor* values, representing changes in matrix dimension and density, respectively. Subfigure (j) displays results for 20 representative matrices from the SuiteSparse collection, ordered on the x-axis by their Gini coefficient (ascending), which correlates with increased workload skew. Empty bars signify out-of-memory (OOM) failures.

sorting. Nevertheless, this penalty diminishes as matrix density increases, underscoring the lightweight nature of AMS in addressing load imbalance.

Turning to the competing methods, a consistent ordering emerges across the sweep. The baseline HASH and ESC algorithms trail their AMS-enhanced counterparts, and the de facto industry solution, Intel MKL, falls behind those baselines in turn: AMS-HYB surpasses the non-sorted MKL by up to 3.83× and the sorted MKLS by up to 8.45×, while AMS-ESC delivers up to 7.66× over MKLS. HEAP and PB rank last, with more unfinished runs owing to their notably larger memory demands. The one exception is PB on highly sparse non-skewed workloads (ER matrices, *edgfactor* 2), where its outer-product-based formulation maximizes data reuse [47], an advantage that quickly diminishes at higher *edgfactor* values and thus implies limited applicability across diverse workloads. These observations

underscore the need for adaptive SpGEMM processing, as no single formulation dominates across the density spectrum.

The SuiteSparse results (Figure 10(j)) reproduce this ordering on real-world sparsity patterns rather than generated ones. AMS-HYB leads every competitor on all 20 matrices, with the single exception of one matrix each against HASH and PB, and the competitors retain their relative standing: the flop-count-based HASH and ESC remain closest, at 1.16× and 1.37× in geometric mean, MKL, MKLS, and HEAP follow at 1.81×, 2.41×, and 2.24×, and PB is furthest behind at 6.08× while also being the only method that fails to complete a matrix. Matching the R-MAT sweep, AMS-HASH and AMS-ESC outperform their flop-count-based counterparts on 19 of the 20 matrices, by up to 47.46% and 25.92%, respectively. That the same ranking holds on matrices we did not generate confirms that the advantage stems from AMS’s scheduling of skewed structure, and not from any property of the R-MAT generators.

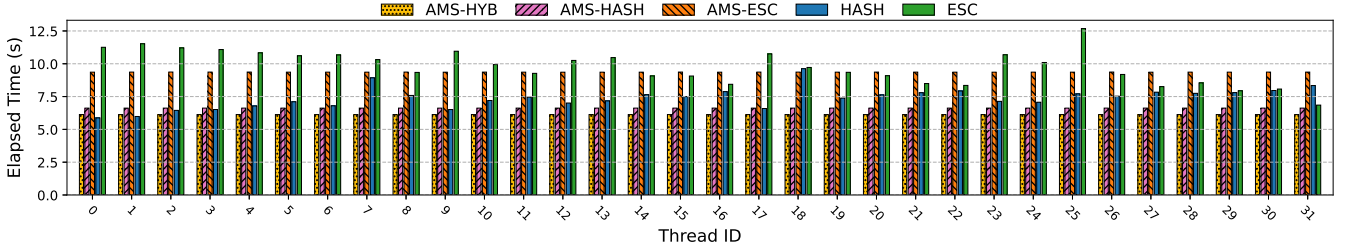


Figure 11: Per-thread elapsed time of the numeric phase comparison of AMS-based variants (AMS-HYB, AMS-HASH, AMS-ESC) and their baseline counterparts (HASH, ESC) on a G500 skewed matrix at *scale* 19 and *edgefactor* 8.

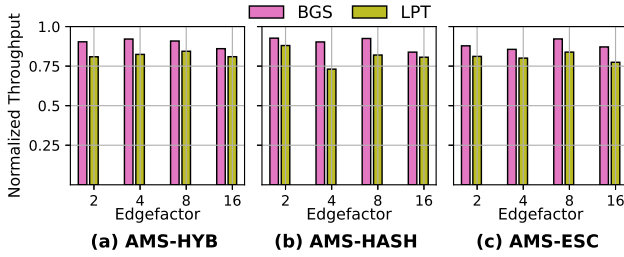


Figure 12: Throughput comparison of BGS and LPT scheduling strategies against CALB. The y-axis shows throughput on G500 (*scale*=19) workloads, normalized to CALB.

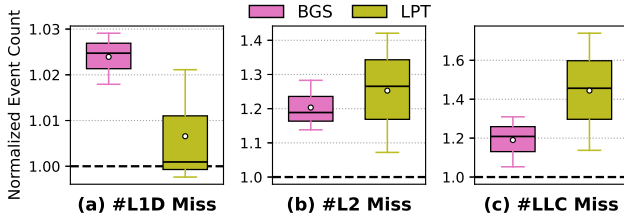


Figure 13: Distribution of the L1D, L2, and LLC misses incurred by the BGS and LPT scheduling strategies on G500 workloads (*scale*=19), each normalized to the corresponding CALB-based execution (dashed line at 1.0). Every box aggregates the samples of all three AMS variants, i.e., AMS-HYB, AMS-HASH, and AMS-ESC. Within each box, the solid line marks the median, the white circle the mean, and the whiskers extend to the most extreme samples within  $1.5\times$  the interquartile range. (a) L1D misses. (b) L2 misses. (c) LLC misses.

### 4.3 Dissecting AMS: CALB and SMTact

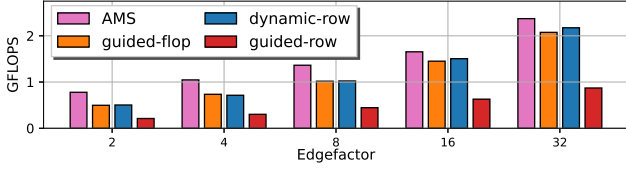
**4.3.1 Effectiveness of CALB.** We now examine the performance benefits of our CALB approach. Designed to alleviate cache contention while maintaining load balance, CALB is evaluated against two alternative scheduling techniques also applicable to our *cf*-sorted morsels: (1) a bidirectional greedy scheduling scheme (BGS), where threads split into two equal-sized groups processing morsels

from opposite ends toward the center, and (2) the classical longest-processing-time-first strategy (LPT) [46], in which all threads start from the high-*cf* end and progress toward the low-*cf* end.

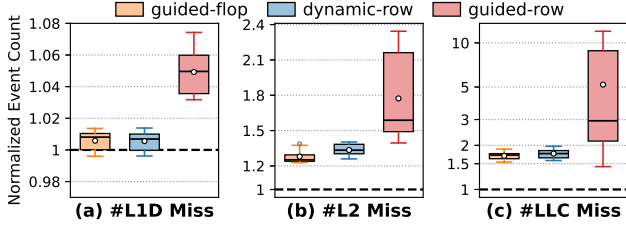
Figure 12 reports the throughput of BGS and LPT, normalized to our CALB method, for three AMS-based algorithms on G500 workloads at *scale* 19. The results show that CALB consistently outperforms both alternatives, achieving 7.90%–19.26% and 8.53%–36.80% higher throughput than BGS and LPT, respectively. While BGS also aims to mitigate contention by avoiding excessive thread concentration on high-*cf* regions, our three-group division strategy in CALB proves more effective at reducing cache pressure, especially at the high-*cf* end. In contrast, LPT performs poorly precisely because it schedules all threads to handle the most demanding, high-contention tasks simultaneously, thereby exacerbating on-chip cache contention.

This explanation rests on cache contention, so we verify it by direct measurement rather than by inference from throughput. Figure 13 profiles the L1D, L2, and LLC misses incurred by BGS and LPT with hardware performance counters, again normalized to CALB on the same G500 workloads, with every box aggregating the samples of all three AMS variants. The counters are collected in runs separate from the timing runs and averaged over the same 10 repetitions used elsewhere; since all three policies execute the same logical SpGEMM work with the same accumulator on the same input, their event counts are directly comparable. The three policies are essentially indistinguishable at the L1D level, where every measurement stays within 3% of CALB (Figure 13(a)). This is expected, since a private L1D is far too small to hold a morsel’s working set under any schedule, so the order in which morsels are dispatched cannot affect it. The divergence emerges deeper in the hierarchy, precisely where concurrently executing morsels begin to share capacity. BGS and LPT incur  $1.14\text{--}1.28\times$  and  $1.07\text{--}1.42\times$  more L2 misses than CALB (Figure 13(b)), and LPT additionally exhibits the highest LLC miss counts and the wider spread of the two (Figure 13(c)). The ordering of the two policies in cache misses therefore mirrors their ordering in throughput, with LPT the worst on both counts, which is what the contention explanation predicts and what a purely load-balance-based account cannot produce.

This validates the rationale behind CALB: by dispersing threads across the *cf* spectrum instead of concentrating them at the high-*cf* end, CALB keeps the aggregate working set of concurrently processed morsels within the cache capacity, confirming that effective load balancing must account for cache contention. Notably, LPT has



**Figure 14: Hashing-based SpGEMM performance on G500 matrices (scale=17) compared to OpenMP dynamic (-row) and guided (-row and -flop) scheduling policies.**



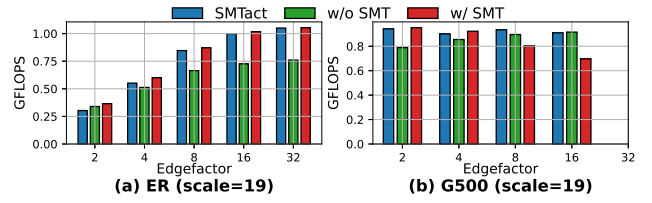
**Figure 15: Distribution of the L1D, L2, and LLC misses incurred by the OpenMP dynamic-row, guided-flop, and guided-row scheduling policies of Figure 14, each normalized to the corresponding AMS-based execution (dashed line at 1.0). Within each box, the solid line marks the median, the white circle marks the mean, and the whiskers extend to the most extreme samples within  $1.5\times$  the interquartile range. Note the logarithmic y-axis in (b) and (c). (a) L1D misses. (b) L2 misses. (c) LLC misses.**

been a seminal scheduling strategy for over 50 years [46], and our findings highlight the need to revisit and refine classical techniques with the idea of cache pressure alleviation.

We further compare AMS against two widely adopted OpenMP load-balancing strategies: dynamic and guided scheduling [18, 89]. These policies schedule a unit of work different from our morsel, so they replace AMS’s scheduling as a whole, and both Figure 14 and Figure 15 are accordingly normalized to AMS. For a fair evaluation, we implemented dynamic scheduling using an empirically determined optimal morsel size of 128 consecutive rows (dynamic-row). Guided scheduling is tested with a minimum morsel set by either 128 rows (guided-row) or a 128-floating-point operation (FLOP) count (guided-flop); both configurations were chosen for the purpose of preserving spatial locality.

As Figure 14 illustrates, AMS achieves higher throughput across all comparisons: 11.87%–48.01% over dynamic-row, 12.89%–47.13% over guided-flop, and a substantial  $2.79\times$ – $3.59\times$  over guided-row. The two margins have different origins, and Figure 15, which profiles the same three policies with hardware performance counters, tells them apart.

Guided-row is the outlier on both figures. OpenMP guided scheduling progressively reduces its chunk size, which, similar to LPT [46], degrades spatial locality during later processing stages, and the profile shows exactly that: its median rises from about  $1.05\times$  at L1D to about  $1.6\times$  at L2 and  $2.9\times$  at the LLC, with individual measurements exceeding  $10\times$  AMS and by far the widest spread of the three policies. This margin is attributable to morsel-wise dynamic scheduling



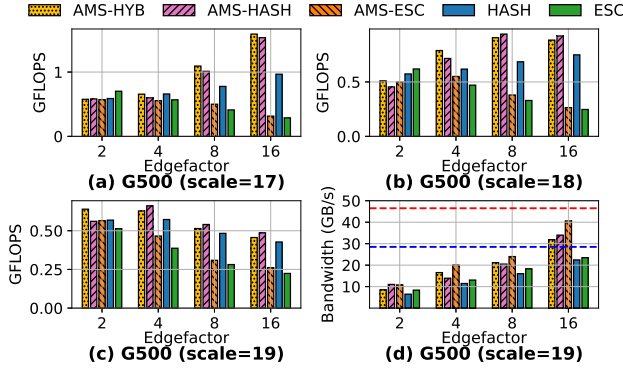
**Figure 16: Effectiveness of SMTact in managing SMT to improve AMS-HYB performance on non-skewed (ER) and skewed (G500) workloads.**

itself (§ 3.1), which keeps consecutive rows together and thereby preserves the locality that guided-row progressively gives up; that alone is enough to outperform guided-row comfortably. This result underscores that spatial locality is a key performance factor, and that it must be preserved together with load balancing.

Dynamic-row and guided-flop avoid this pitfall, as their work units keep consecutive rows together and they distribute load as effectively as AMS does. Yet both still trail AMS, and the cache profiles show where the difference arises. At the private L1D, where a difference in spatial locality would first become visible, both remain within roughly 1% of AMS. The policies diverge only at the levels that concurrently executing morsels share, with median counts rising to  $1.27$ – $1.34\times$  at L2 and to  $1.7$ – $1.8\times$  at the LLC, an ordering that matches their relative throughput in Figure 14. Since locality is comparable across the three, the remaining source of this gap is inter-thread cache contention, which CALB alleviates by co-scheduling high-*cf* and low-*cf* morsels. We therefore attribute the advantage of AMS over these two policies to CALB, and conclude that contention-aware scheduling is essential for effective parallel load balancing.

**4.3.2 Effectiveness of SMTact.** Our SMTact is designed as an adaptive SMT-enablement strategy by balancing the trade-off between increased computational capability and elevated cache pressure. To evaluate its effectiveness, we compare SMTact against two fixed configurations: SMT always enabled (w/ SMT) and SMT always disabled (w/o SMT). The results, presented in Figure 16, include both non-skewed and skewed workloads. As illustrated, neither fixed configuration is uniformly viable: w/ SMT falls up to 23.9% short of the best configuration on the skewed G500 workloads, where the extra logical threads aggravate cache contention, whereas w/o SMT forfeits up to 28.6% on the non-skewed ER workloads that profit from the additional thread-level parallelism.

Since SMTact ultimately executes as one of these two configurations, its goal is not to surpass both in every case, but to automatically select a configuration at or close to the best. In our experiments, SMTact stays within 8.2% of the best fixed configuration in eight of the nine cases. The exception is the extremely sparse ER matrix at *edgefactor* 2, where SMTact falls 17.1% short of the best fixed configuration because the sorting overhead introduced by CALB is not yet amortized; this penalty is quickly offset as *edgefactor* or matrix scale increases. Given the wide variability in sparsity patterns in real-world SpGEMM workloads, SMTact thus offers a robust default that avoids the substantial degradation that either fixed policy incurs outside its sweet spot.



**Figure 17: Performance of SpGEMM on G500 workloads. (a)–(c): Measured throughput at G500 scales 17, 18, and 19. (d): Sustained bandwidth measured for different algorithms for the scale-19 workload (the red and blue dashed lines indicate the peak bandwidth of the CXL-augmented memory system (46.50 GB/s) and host DRAM (28.50 GB/s), respectively).**

#### 4.4 Evaluation on Bandwidth-Expanded Systems

We now evaluate AMS on a bandwidth-expanded memory system, where CXL boosts aggregate bandwidth but introduces higher access latency. As discussed in § 1, load imbalance can inadvertently amplify CXL’s latency penalty on straggler threads, negating the bandwidth gain and making effective load balancing particularly critical. For this evaluation, the machine has the same CPU and OS configuration as our previous platform but features host DRAM of 28.50 GB/s bandwidth. Additionally, a CXL expansion memory of 20.95 GB/s bandwidth is installed, yielding a combined system bandwidth of 46.50 GB/s [58, 70, 118].

We evaluate AMS approaches against their vanilla counterparts using skewed (G500) workloads and report the results in Figure 17. As shown, AMS-based methods generally outperform the other methods (Figure 17(a)–(c)). Furthermore, the profiling results confirm that AMS effectively utilizes the additional CXL-augmented bandwidth: AMS-ESC sustains a bandwidth of up to 40.60 GB/s, far exceeding the DRAM-only peak (28.50 GB/s) (Figure 17(d)). In contrast, the bandwidth consumption of the vanilla counterparts does not exceed that of the host DRAM, indicating that AMS indeed improves bandwidth utilization by addressing load imbalance.

We also observe an interesting trend in Figure 17(a)–(c): while AMS-based methods remain competitive on skewed workloads, the performance gap between AMS and the vanilla baselines differs for G500 matrices at *scale* 17 versus those at *scales* 18 and 19. Taking the HASH-based variants as an example, AMS-HASH outperforms HASH by 58.77% on the G500 matrix at *scale* 17, delivering 1.53 GFLOPS versus 0.97 GFLOPS (*edgefactor* 16). However, this advantage diminishes to 23.16% at *scale* 18, with a similar trend observed at *scale* 19.

This reduction in the performance gap is mainly attributed to the coincidental load balance occasionally achieved by the static flop-count-based scheme. It is the extent of this imbalance that

largely determines the performance degradation of baselines relative to their AMS counterparts. As Figure 4 illustrates, computation and materialization times can exhibit opposing trends across threads. This is precisely the multi-dimensional skew identified in § 2.2, namely that computation and materialization costs trend in opposite directions, and that any single-metric scheme can only coincidentally balance them. In some cases, these opposing patterns may randomly offset each other, resulting in a seemingly balanced overall execution time despite the underlying skew. This balance, however, is merely incidental and heavily dependent on the platform’s relative sensitivity to computation versus memory access. Consequently, it fails to provide consistent or portable load balance across different architectures.

Previous results on a 179.50 GB/s bandwidth memory corroborate this explanation (see Figure 10(g)–(i)). With this higher bandwidth, AMS-HASH achieves a 45.44% performance increase over HASH (1.52 GFLOPS vs. 1.05 GFLOPS) on the G500 matrix at *scale* 18, because the increased bandwidth significantly reduces the platform’s vulnerability to materialization cost. Collectively, these results validate our claim that AMS provides a robust and portable solution to load balancing across diverse systems.

#### 4.5 Evaluation on Other Sparse Matrix Kernels

Having established AMS’s portability across memory systems, we now turn to its generalizability across kernels. Beyond SpGEMM, we evaluate AMS on other sparse matrix multiplication kernels, specifically, sparse matrix-dense matrix multiplication (SpMM) and sparse matrix-vector multiplication (SpMV). Since these kernels produce fixed-size dense output, we form morsels by grouping consecutive rows based on a targeted *nnz* count in the left-hand side matrix sub-block (see § 3.5), with each morsel sized at half the L2 cache capacity. The resulting morsels have disparate densities, enabling CALB and SMTact to alleviate cache contention as in SpGEMM.

We compare this AMS-based SpMM and SpMV implementation against the state-of-the-art Intel MKL library on both non-skewed ER and skewed G500 workloads, running on the same main-memory system as our default platform (§ 4.1), taking ER at *scale* 21 and G500 at *scale* 20 and sweeping the *edgefactor* from 2 to 64, so that the behavior at higher densities remains visible.

As shown in Figure 18(a)–(d), our method achieves on-par performance with MKL on non-skewed ER workloads while significantly outperforming it on skewed G500 workloads, with throughput gains of up to 57.25% and 3.51× for SpMM and SpMV, respectively. Figure 18(e)–(h) reports the sustained memory bandwidth of the same runs and shows where that gain originates: on the non-skewed ER workloads the two methods sustain comparable bandwidth, with SpMM approaching the 179.50 GB/s physical limit of local DRAM, whereas on the skewed G500 workloads AMS sustains substantially higher bandwidth than MKL at every *edgefactor*. This confirms that the advantage comes from the load balancing of AMS together with the on-chip cache contention it alleviates, exactly as in SpGEMM, where stragglers induced by skew leave bandwidth idle and concurrently executing dense morsels evict each other’s working sets. By removing both, AMS converts bandwidth that the baseline wastes

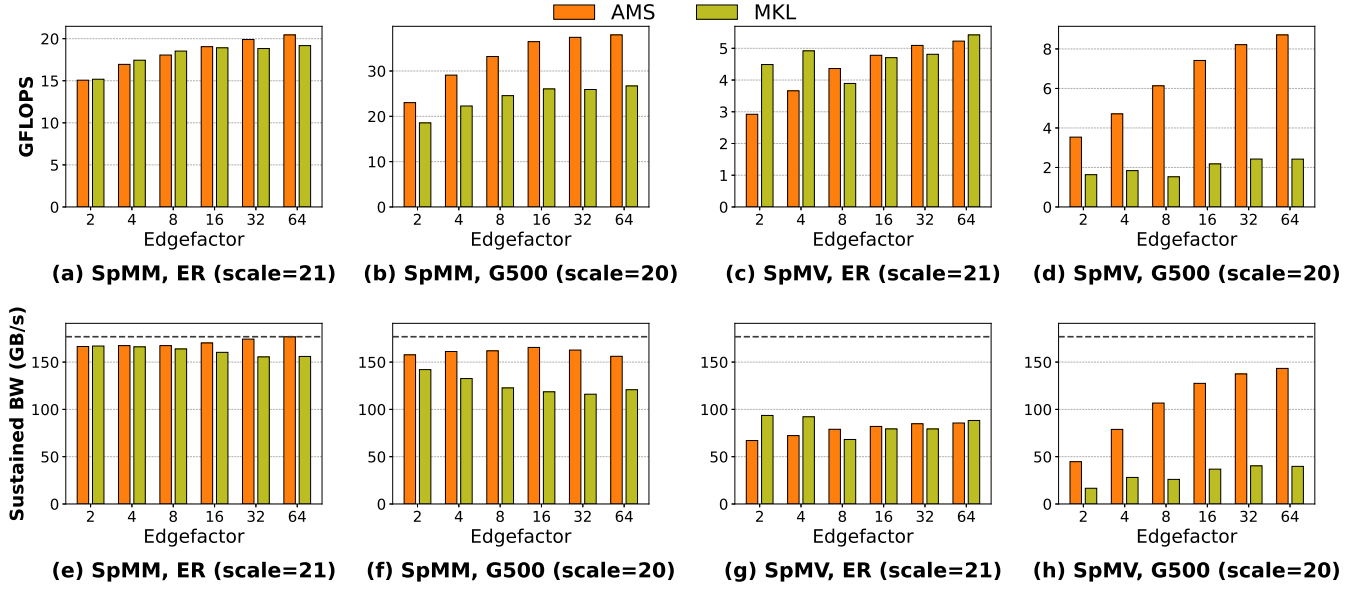


Figure 18: Performance of AMS versus MKL for SpMM and SpMV operations on ER and G500 workloads, reported as throughput in (a)–(d) and as sustained memory bandwidth in (e)–(h). The dashed line marks the 179.50 GB/s physical bandwidth of our platform’s local DRAM.

into useful work, which is what drives the throughput gain on these memory-bound kernels.

Comparing the two kernels is instructive in its own right. SpMM sustains markedly higher bandwidth than SpMV throughout, as its dense right-hand operand offers a reuse that a single vector cannot, so SpMV wastes a far larger share of the available bandwidth, and the waste is worst under skew, where MKL sustains only a small fraction of the DRAM peak (Figure 18(h)). AMS largely remedies this shortcoming, lifting the sustained bandwidth of skewed SpMV to roughly three times that of MKL and, at the higher *edgefactors*, into the range that SpMM attains. This underscores that, in such memory-bound kernels, skew-aware scheduling improves performance by recovering wasted memory bandwidth rather than by reducing arithmetic work. Together, these results demonstrate the broad applicability and robust advantages of AMS’s core scheduling principles across various sparse matrix multiplication kernels.

#### 4.6 Join-Aggregate and Join-Project Query Evaluation

We now validate AMS on relational workloads, where modeling tables as sparse matrices allows SpGEMM to evaluate join-aggregate and join-project queries without materializing the intermediate join result [1, 3, 63], a key advantage for many-to-many and self-joins that produce large intermediate cardinalities. We evaluate two representative relational query classes on the same datasets, the join-aggregate query and the join-project query, where an aggregation or a projection is applied on top of the join operation, which is the most direct way existing studies exploit matrix multiplication to accelerate relational workloads [33, 59, 102].

We employ widely used datasets from prior works [33, 59, 102], and also evaluate on an industrial OLAP ClickHouse benchmark [97] with two sampled workloads: CLH0, a uniform 5% sample of users, and CLH1, the top 5% of users by pageviews (see Table 3 for key statistics). These datasets span a wide range of skewness, from mildly to highly skewed, as quantified by the Gini coefficient in Table 3. Notably, the uniform sample exhibits a higher Gini coefficient than the top-user sample (0.59 versus 0.39), as restricting the population to the heaviest users truncates the long tail, whereas a representative sample preserves the skew inherent in real-world workloads. As for the query form, we perform self-join (i.e.,  $R = S$ ) with aggregation in the form:

```
SELECT R.user, S.user, SUM(R.val * S.val)
FROM R, S
WHERE R.item = S.item AND R.user <> S.user
GROUP BY R.user, S.user
```

The corresponding join-project query replaces the aggregation with a projection over the same self-join:

```
SELECT DISTINCT R.user, S.user
FROM R, S
WHERE R.item = S.item AND R.user <> S.user
```

Both queries measure user-wise similarity, where “item” and “val” abstract the shared attribute and measure across scenarios, e.g., URL/click-count in ClickHouse (CLH0 and CLH1) and movie/rating in MovieLens (MVLN). The aggregate form is particularly challenging, as the aggregated values increase the demand for memory, and it is not amenable to existing aggregate query rewriting techniques [25, 36–38, 81, 115, 116, 122, 123].

Relational join queries impose no ordering on their output by default, whereas producing row-wise sorted output accounts for a substantial share of the total cost, as § 4.2 shows and prior SpGEMM

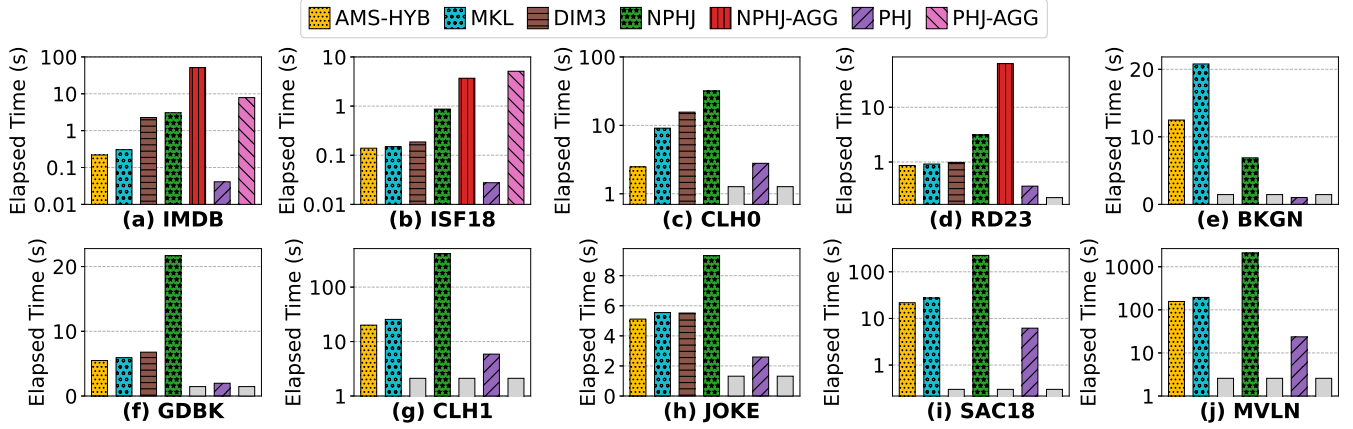


Figure 19: Execution time comparison across diverse join-aggregate workloads; gray bars with “N/A” denote algorithms that run out of memory on those specific workloads; note that vertical axes use a logarithmic scale to capture the wide range of execution times, except for subfigures (e), (f), and (h).

Table 3: Summary of the 10 relational workloads used for both join-aggregate and join-project evaluation.

| Dataset    | $ user $ | $ tuple $ | Gini   | $ OUT_J $ | $ OUT_A $ | $ OUT_J / OUT_A $ |
|------------|----------|-----------|--------|-----------|-----------|-------------------|
| IMDB [60]  | 733.47K  | 7.52M     | 0.3453 | 214.77M   | 176.91M   | 1.2140            |
| ISF18 [85] | 1.82K    | 1.03M     | 0.5586 | 327.58M   | 3.29M     | 99.6586           |
| CLH0 [97]  | 850.00K  | 2.90M     | 0.5858 | 2.43B     | 2.34B     | 1.0415            |
| RD23 [100] | 6.72K    | 3.91M     | 0.5276 | 4.61B     | 45.02M    | 102.3613          |
| BKGN [68]  | 350.33K  | 5.15M     | 0.7242 | 11.43B    | 6.77B     | 1.6892            |
| GDBK [126] | 53.42K   | 5.98M     | 0.1293 | 19.63B    | 2.51B     | 7.8261            |
| CLH1 [97]  | 850.00K  | 24.05M    | 0.3862 | 20.34B    | 11.41B    | 1.7823            |
| JOKE [59]  | 23.50K   | 1.71M     | 0.1709 | 32.65B    | 552.25M   | 59.1219           |
| SAC18 [67] | 104.66K  | 10.00M    | 0.6988 | 68.52B    | 5.94B     | 11.5325           |
| MVLN [59]  | 138.49K  | 20.00M    | 0.5807 | 269.61B   | 16.80B    | 16.0477           |

- $|user|$  and  $|tuple|$  are equivalent to the matrix row count and total nnz count, respectively, reflecting the relational-to-sparse-matrix duality; Gini captures the user-wise nnz distribution [69].
- $|OUT_J|$  and  $|OUT_A|$  represent the cardinalities of the join result and the final aggregated result, respectively.

studies report [83, 84]. We therefore conduct this evaluation in the non-sorted regime, comparing AMS-HYB with: (1) DIM3 [59], the state-of-the-art join-aggregate method based on SpGEMM; (2) NPHJ-AGG, where a fast hash table [98] performs aggregation on-the-fly alongside the probing phase of non-partitioned hash join (NPHJ) [12, 96], also bypassing intermediate materialization; (3) PHJ-AGG, where we use the cache-efficient partitioned hash join [8–10, 19] to perform the join and materialize the intermediate result, and after that we use the same efficient hash table [98] as NPHJ-AGG to perform the aggregation<sup>19</sup>; (4) the industrial SpGEMM method: MKL with non-sorted output. We refer to NPHJ-AGG and PHJ-AGG collectively as the “join-then-aggregate” methods, as both evaluate the query in a traditional way, performing the aggregation on top of an explicit join rather than fusing the two as the matrix-based methods do. For reference, we also include NPHJ and PHJ as join-only baselines (which do not materialize the intermediate

<sup>19</sup>We do not have PHJ-AGG perform the on-the-fly aggregation as NPHJ-AGG, because on-the-fly aggregation would incur substantial memory traffic to the aggregation hash table, breaking the cache efficiency that partitioning is designed to deliver [9, 10, 80].

Table 4: Summary of all evaluated join-aggregate query processing approaches

| Method                                                             | In-place Agg.* | Description                                                                                                  |
|--------------------------------------------------------------------|----------------|--------------------------------------------------------------------------------------------------------------|
| <i>Traditional join followed by aggregation</i>                    |                |                                                                                                              |
| PHJ-AGG [96, 98]                                                   | ✗              | Radix partitioned hash join with aggregation; deduplication via fast hash table [59, 98].                    |
| NPHJ-AGG [15, 98]                                                  | ✓              | Non-partitioned hash join; aggregates computed during probing to skip the final deduplication pass [59, 98]. |
| <i>Join-alone algorithms (no aggregation; for comparison only)</i> |                |                                                                                                              |
| PHJ [96]                                                           | —              | Radix partitioned hash join; serves as the join backend of PHJ-AGG. Does not produce aggregation results.    |
| NPHJ [15]                                                          | —              | Non-partitioned hash join; serves as the join backend of NPHJ-AGG. Does not produce aggregation results.     |
| <i>State-of-the-art join-aggregate techniques</i>                  |                |                                                                                                              |
| DIM3 [59]                                                          | ✓              | Hybrid sparse/dense join-aggregate query algorithm; intersection-free partitioning eliminates deduplication. |
| <i>Industry SpGEMM competitors</i>                                 |                |                                                                                                              |
| MKL [61]                                                           | ✓              | Intel oneMKL SpGEMM (non-sorted variant); aggregation replaces deduplication inside the accumulator.         |
| <i>Proposed AMS-based approaches</i>                               |                |                                                                                                              |
| AMS-HYB                                                            | ✓              | AMS scheduling with hybrid HASH/ESC selection; eliminates the separate deduplication pass.                   |

\* In-place Agg. denotes whether aggregation is performed on-the-fly alongside the join, or both operations are directly merged into one.

results). All evaluated methods are summarized in Table 4 for a clear reference.

As for the join-project setting, we use the same set of methods but replace the aggregation with a projection. A projection no longer references the value domain, so both the matrix-based and the traditional “join-then-project” methods can omit the value computation altogether, saving the memory and the arithmetic that the values would otherwise incur, and every method that produces the query output is correspondingly faster than in the join-aggregate setting above. For the matrix-based methods, i.e., AMS-HYB, MKL, and DIM3, the value array of the output CSR matrix is thus neither

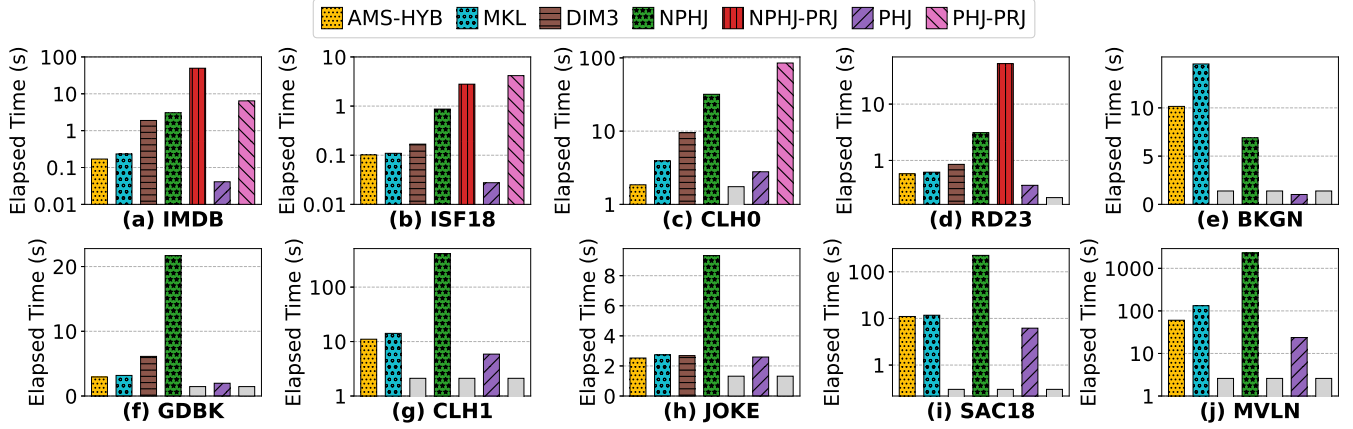


Figure 20: Execution time comparison across diverse join-project workloads; gray bars with “N/A” denote algorithms that run out of memory on those specific workloads; note that vertical axes use a logarithmic scale to capture the wide range of execution times, except for subfigures (e), (f), and (h).

Table 5: Tuple-to-CSR conversion cost, covering both operands of each query, and its share of the total AMS-HYB runtime under the two query classes.

| Dataset | Conversion (s) | Share (%)      |              |
|---------|----------------|----------------|--------------|
|         |                | Join-Aggregate | Join-Project |
| IMDB    | 0.0126         | 5.7536         | 7.3995       |
| ISF18   | 0.0043         | 3.0764         | 4.1958       |
| CLH0    | 0.0087         | 0.3534         | 0.4729       |
| RD23    | 0.0064         | 0.7476         | 1.1090       |
| BKGN    | 0.0128         | 0.1025         | 0.1260       |
| GDBK    | 0.0097         | 0.1764         | 0.3282       |
| CLH1    | 0.0679         | 0.3391         | 0.6139       |
| JOKE    | 0.0049         | 0.0962         | 0.1948       |
| SAC18   | 0.0156         | 0.0720         | 0.1422       |
| MVLN    | 0.0488         | 0.0313         | 0.0807       |

allocated nor populated, and the projection is read directly from the sparsity structure of the matrix product. Moreover, following prior works [3, 59], we apply an early termination mechanism to AMS-HYB and DIM3, which abandons the accumulation of an output cell once it is known to be nonzero, as join-project is a Boolean matrix product in which only the existence of each output nonzero matters, whereas the join-aggregate form must sum every contribution. It should be noted that we are not able to apply this mechanism to MKL, as the proprietary library exposes only a numerical product. For the traditional “join-then-project” methods, denoted NPHJ-PRJ and PHJ-PRJ, no value payload is carried or combined, and the two mirror their aggregate counterparts: NPHJ-PRJ projects on-the-fly alongside probing as NPHJ-AGG does, whereas PHJ-PRJ projects only after the intermediate result is materialized, for the same reason as PHJ-AGG.

It should be noted that in this evaluation, the table inputs are in the original row-wise tuple format  $\langle \text{row-index}, \text{column-index},$

value), e.g.,  $\langle \text{user}, \text{URL}, \text{click count} \rangle$  for the ClickHouse workloads, which is equivalent to the coordinate format (COO) in sparse matrix storage [39, 66, 94]<sup>20</sup>. The hash-join methods operate on these tuples directly, whereas the matrix-based methods require CSR and therefore incur a preliminary pass of data format conversion. We perform this tuple-to-CSR conversion with a parallel pass of histogram, prefix sum, and scatter, following the common practice of mainstream sparse-matrix libraries [48, 112], and charge its time to all matrix-based methods. Table 5 reports this cost on every workload, covering both operands of each query. The conversion costs at most 7.40% of the AMS-HYB runtime, with the two largest shares on IMDB and ISF18, which have the lowest  $|OUT_j|/|tuple|$  ratios and therefore the least multiplication over which to amortize the linear pass. On the remaining eight workloads it stays below 1.2% under both query classes, so the input format is not a performance concern for the matrix-based methods.

Figure 19 presents the join-aggregate results. As shown, AMS-HYB consistently achieves the best performance among all aggregate variants, demonstrating its effectiveness in addressing relational join queries. Surprisingly, even compared with the join-only baseline NPHJ, AMS-HYB wins 9 out of 10 cases, losing only on BKGN, demonstrating the superiority of SpGEMM methods in handling joins with large output cardinalities. Comparing the two runners-up, MKL and DIM3, DIM3 is the weaker of the two, trailing MKL by 1.65 $\times$  in geometric mean over the workloads both complete, and it cannot finish on 4 workloads (BKGN, CLH1, SAC18, and MVLN) due to out-of-memory errors. This is because DIM3 uses an “estimate-partition-multiplication” approach that tends to over-estimate output size under skew, a common scenario in real-world workloads, and thus incurs significant memory materialization cost. Moreover, DIM3 and other prior matrix-based works [33, 54, 55, 102] mainly target an algorithmic remapping, i.e.,

<sup>20</sup>For join-project, only existence matters, so a recorded pair is the nonzero entry itself and the row-wise tuple can drop the value, leaving  $\langle \text{row-index}, \text{column-index} \rangle$  alone as exactly the COO of a sparse Boolean matrix.

expressing the query as a sparse matrix product, and leave the parallelization to a plain OpenMP loop without a scheduling design of their own. DIM3 therefore inherits exactly the load imbalance that AMS targets, which is why it generally trails MKL, an industrial standard with mature thread scalability, even where DIM3 completes. Notably, AMS-HYB leads MKL by 1.37 $\times$  in geometric mean and by up to 3.69 $\times$  on CLH0, a skewed workload (Gini of 0.59) at a sizable matrix dimension (850K rows), confirming that AMS’s ability to address multi-dimensional skew and alleviate on-chip cache contention translates directly to relational workloads where skew is prevalent. The gain concentrates on the large and skewed workloads, where load imbalance and cache contention, if not handled properly, inflate memory access cost enough to dominate runtime, whereas the smaller instances stay largely cache-resident and are thus less prone to load imbalance or excessive memory access cost <sup>21</sup>.

Turning to the hash-join baselines, NPHJ-AGG and PHJ-AGG finish on only three and two of the ten workloads, respectively, in both cases the smallest instances. This is because, regardless of whether the aggregation is performed on-the-fly or after the join result is fully populated, the hash table must grow dynamically to hold the unhashed entries that are to be aggregated. This dynamically growing hash table is usually resized with a specific scaling factor (typically 2 $\times$ ), and therefore exhausts the memory footprint very quickly. Beyond memory limitations, comparing the “join-then-aggregate” approaches with their respective join-only baselines, we can see that the aggregation-based methods are more than an order of magnitude slower on average, indicating the significant cost of aggregation, and thus underscoring the necessity and importance of SpGEMM-based methods. This is particularly true for PHJ and PHJ-AGG: PHJ alone achieves the best runtime on all workloads but CLH0, yet aggregation inflates its runtime by more than two orders of magnitude wherever PHJ-AGG completes (for instance from 0.041 to 7.852 seconds on IMDB). This stark contrast indicates that the aggregation operation itself is the dominant bottleneck, underscoring the need for effective join-aggregate query processing.

We now turn to the join-project setting, which shares the same joins, datasets, and skew. Figure 20 reports this second setting, and the ranking of the methods carries over unchanged. AMS-HYB again achieves the best performance among all join-project variants. MKL again follows as the closest competitor, trailing by 1.33 $\times$  in geometric mean and by up to 2.19 $\times$  on MVLN. DIM3 again places third, running 2.64 $\times$  slower than AMS-HYB in geometric mean over the workloads it completes, a wider gap than the 2.28 $\times$  it exhibits under aggregation, and it fails on the same four workloads with the largest outputs, for the same two reasons given above.

Compared with the join-aggregate results, the join-project workloads generally exhibit lower runtime, as the query no longer requires the value computation and thereby incurs less arithmetic and less memory traffic. AMS-HYB, DIM3, and MKL accordingly run 1.66 $\times$ , 1.34 $\times$ , and 1.71 $\times$  faster in geometric mean than their respective join-aggregate counterparts. DIM3, however, still falls behind MKL by 2.10 $\times$  in geometric mean over the workloads both

**Table 6:  $M_l$  construction on DBLP V10 (seconds).**

| Meta-path | row     | nnz    | Gini | AMS-HYB | MKL   | Speedup       |
|-----------|---------|--------|------|---------|-------|---------------|
| APCPA     | 1.77M   | 4.98M  | 0.62 | 0.052   | 0.117 | 2.24 $\times$ |
| CPAPC     | 5.08K   | 4.98M  | 0.78 | 0.067   | 0.120 | 1.80 $\times$ |
| APTPA     | 1.77M   | 48.48M | 0.60 | 0.158   | 0.529 | 3.34 $\times$ |
| TPAPT     | 120.69K | 48.48M | 0.94 | 0.235   | 0.382 | 1.62 $\times$ |
| Geomean   | –       | –      | –    | 0.107   | 0.231 | 2.16 $\times$ |

complete, losing on five of the six, despite being the only one of the two that applies early termination. This underscores that, without proper scheduling and load balancing, performing less work does not always translate into higher performance. AMS-HYB, in contrast, pairs early termination with contention-aware scheduling and load balancing, and therefore converts the reduced work into an actual runtime gain.

As for the “join-then-project” methods, they behave as their aggregate counterparts do. NPHJ-PRJ and PHJ-PRJ each complete on only three of the ten workloads, since, whether the projection is performed on-the-fly or after the join result is fully populated, its hash table must grow dynamically to hold unhashed entries and typically doubles in size until memory is exhausted, and NPHJ-PRJ remains up to 17 $\times$  slower than the join-only NPHJ, confirming that the projection, and not the join, is the dominant cost. AMS-HYB again wins 9 of the 10 cases against NPHJ itself, losing only on BKGn, which reaffirms the superiority of SpGEMM-based methods on relational join-project queries.

## 4.7 Graph-Analytic Evaluation

Real-world graph workloads are typically sparse, and are commonly modeled as sparse matrices through their adjacency representation, which places one nonzero at every edge and under which advancing a set of vertices to their neighbors becomes a matrix product. SpGEMM is therefore an important building block of graph analytics, expressing operators such as frontier expansion, neighborhood composition, and masked edge enumeration as a single kernel. Such graphs are moreover overwhelmingly power-law in practice, so their adjacency matrices carry exactly the row-wise skew that AMS is designed for, and a contention-aware SpGEMM benefits these workloads immediately. We evaluate four representative graph-analytic workloads, namely meta-path construction for PathSim similarity search, breadth-first search, triangle counting, and an end-to-end Markov clustering pipeline.

**4.7.1 PathSim Meta-Path Construction.** We first evaluate AMS on PathSim [104], a meta-path-based similarity measure for heterogeneous information networks, focusing on the intermediate matrices it must construct. Given a symmetric meta-path, PathSim scores node pairs through a commuting matrix  $M = M_l M_l^T$ , whose left half  $M_l$  is a chain of sparse matrix products. Constructing  $M_l$  is therefore an SpGEMM workload, and it dominates the cost of answering a query.

We evaluate on the AMiner DBLP V10 citation network [107], comprising 1.77M authors (A), 3.08M papers (P), 5,078 venues (C), and 120,685 terms (T), with 9.48M author-paper, 2.57M venue-paper, and 22.64M paper-term edges. We select four symmetric meta-paths

<sup>21</sup> A more detailed analysis of the performance gap between AMS-HYB and MKL is given in Appendix C.

that place each node type in the row dimension: APCPA and APTPA relate authors through shared venues and terms, whereas TPAPT and CPAPC relate terms and venues through shared authors. As Table 6 summarizes, they differ widely in both shape and skew, and form two transpose pairs (APCPA/CPAPC and APTPA/TPAPT) that perform the same work over row counts differing by  $350\times$  and  $15\times$ . All four exhibit pronounced row-wise skew (Gini 0.60–0.94), reflecting prevalent skew in real-world graphs. We compare AMS-HYB against the industry-standard MKL and report the SpGEMM time of the  $M_I$  construction.

As reported in Table 6, AMS-HYB outperforms MKL on all four meta-paths, by  $1.62\times$  to  $3.34\times$ , giving a geometric-mean speedup of  $2.16\times$ . These gains span a  $350\times$  range of row counts, from the 5,078-row CPAPC to the 1.77M-row APTPA, indicating that the benefit of AMS does not depend on a particular matrix shape. Within each transpose pair, however, the gain is consistently larger for the variant with more rows ( $2.24\times$  against  $1.80\times$ , and  $3.34\times$  against  $1.62\times$ ), as a larger row count yields finer morsel granularity for CALB to redistribute and better amortizes the cost of *cf*-based morsel sorting. Overall, AMS extends its advantage beyond matrix squaring and relational queries to meta-path construction on heterogeneous networks.

**4.7.2 Breadth-First Search.** We next evaluate AMS on multi-source breadth-first search, the traversal primitive on which a large family of graph-analytic tasks is built [34]. A notable application is betweenness centrality, which scores a vertex by the fraction of shortest paths that pass through it. It obtains those shortest-path counts by running one breadth-first traversal from every source vertex, so its cost is essentially that of many independent traversals over a shared graph. Other representative use cases include connected components, reachability and *k*-hop neighborhood queries, unweighted single-source shortest paths, and diameter or eccentricity estimation, each of which rests on the same primitive and therefore benefits directly from a faster traversal step.

SpGEMM is exactly the kernel that carries out one traversal level in the linear-algebraic formulation [84]. Its left operand is the square adjacency matrix of the graph; its right operand is a tall-skinny matrix holding one column per source, each column a sparse vector that marks the vertices that source has reached at the current level. Under the row-wise formulation of § 2.1, output row *i* gathers the frontier rows indexed by the nonzeros of row *i* of the adjacency operand and accumulates them, so the degree of vertex *i* sets how many frontier rows that row must gather, and a single product advances every source by one hop. Multi-source BFS is therefore a sequence of such square-by-tall-skinny products, each of which carries the power-law degree distribution of the graph directly into the row-wise skew that AMS is built to absorb.

We follow the setup of Nagasaka et al. [84], who cast this kernel as the product of a square graph matrix with a tall-skinny frontier matrix and evaluate it on synthetic G500 graphs, which preserve the power-law degree distribution common to real-world graphs while allowing the frontier width to be swept independently of graph size and density. We sweep the graph *scale* over 17–19 and the number of concurrent sources over  $2^{10}$ – $2^{16}$ , and extend that setup by sweeping the *edgefactor* over 2–32 as well, so that density varies alongside frontier width. As frontier expansion imposes no

ordering requirement on its output, a frontier being consumed as a set of reached vertices, we report the non-sorted methods only, comparing AMS-HYB and AMS-HASH against the HASH baseline and the non-sorted Intel MKL variant.

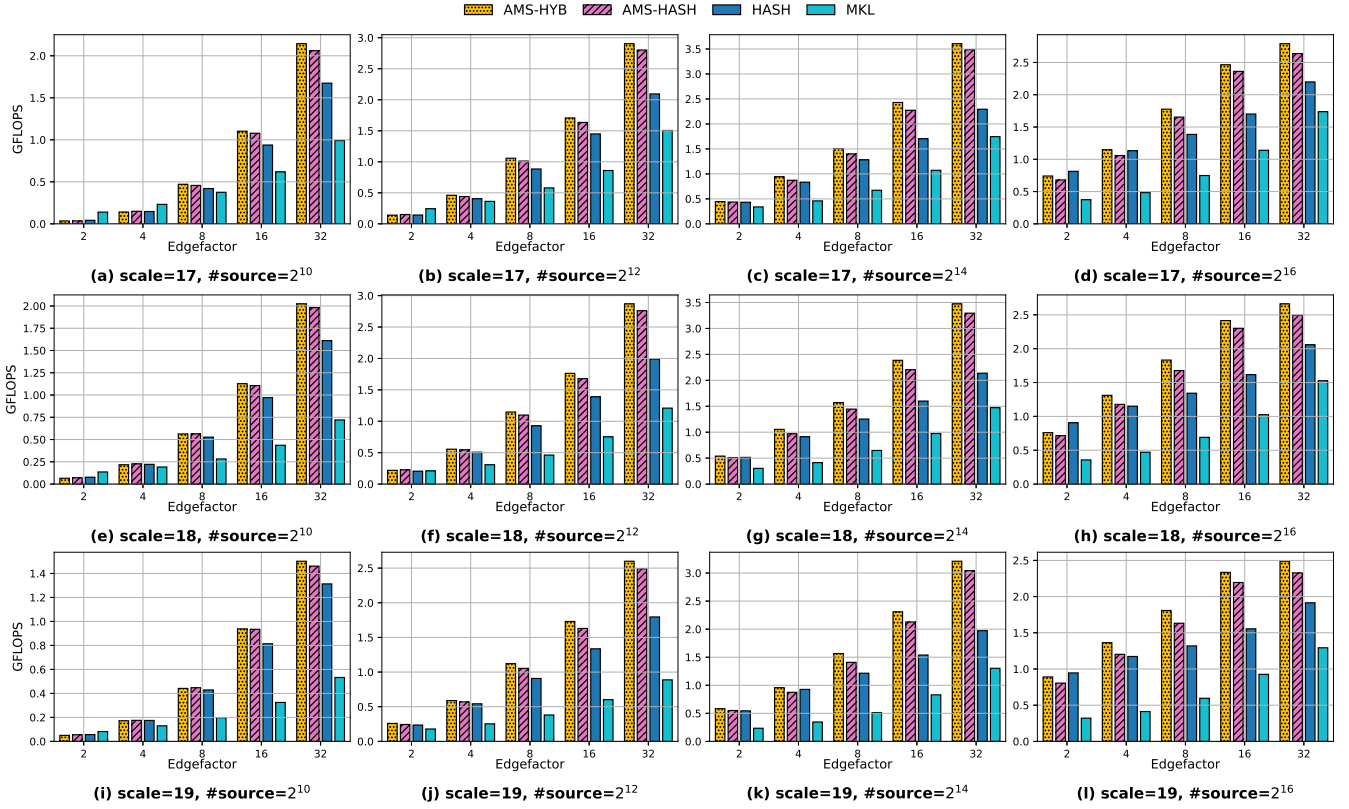
Figure 21 shows the results, and we can see that across all evaluated configurations, AMS-HYB attains the highest throughput of the four implementations in 47, and it leads the HASH baseline by  $1.18\times$  in geometric mean and by up to  $1.63\times$ , ahead in 50 of them, while AMS-HASH leads by  $1.13\times$ ; against MKL the margin is far wider,  $1.91\times$  in geometric mean and up to  $3.31\times$ . The 13 configurations AMS-HYB does not win all lie at the edges of the sweep. MKL takes 5 of them at the smallest and sparsest points, *edgefactor* 2–4 with  $2^{10}$ – $2^{12}$  sources, where the product is too small for AMS’s morsel preparation to be amortized; the HASH baseline takes 3, all at *edgefactor* 2 with the widest frontier; and AMS-HASH takes the remaining 5 by margins below 5%, 4 of them at  $2^{10}$  sources.

Two trends are visible across the panels. The advantage grows monotonically with density, from  $0.96\times$  at *edgefactor* 2 to  $1.07\times$ ,  $1.21\times$ ,  $1.33\times$ , and  $1.38\times$  at *edgefactor* 32, and the single regime in which AMS does not pay off is the sparsest one, where the *cf*-based morsel sorting is not yet amortized, exactly as observed for matrix squaring in § 4.2. The advantage also grows with the number of concurrent sources, from  $1.05\times$  at  $2^{10}$  to  $1.20\times$  at  $2^{16}$ , because a wider frontier makes each traversal level denser and thus concentrates more contention for CALB to relieve. Both trends confirm that the gains follow the amount of skew-induced contention actually present, rather than being an artifact of any particular graph size.

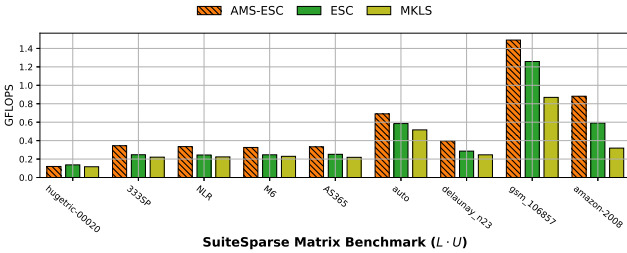
**4.7.3 Triangle Counting.** We then evaluate AMS on triangle counting [106], which reports the number of triangles a graph contains and thereby underpins the clustering coefficient and the transitivity ratio, as well as community detection and spam or anomaly detection on web and social networks. In practice, triangle counting is typically defined on an undirected graph, which can therefore be modeled as a square symmetric sparse adjacency matrix. The most common formulation multiplies the strictly lower and upper triangular parts of that matrix,  $L \cdot U$ , which enumerates the wedges of the graph, that is, its paths of length two, and masks the product with the adjacency matrix so that only the wedges closing into a triangle are retained. The masking step merges each output row with the corresponding adjacency row, and therefore demands row-wise ordered output.

Following the setup of Nagasaka et al. [84], we evaluate triangle counting on real-world square symmetric matrices. Specifically, we use the nine matrices of Table 1 that are square and structurally symmetric, reported in Figure 22, with each prepared in the way the pipeline prescribes, where its rows are reordered by increasing nonzero count and the reordered matrix is then split into the triangular operands *L* and *U*. Since the masked formulation requires sorted output, we report the sorted methods only, comparing AMS-ESC against the ESC baseline and the sorted Intel MKL variant (MKLS).

As Figure 22 shows, AMS-ESC outperforms the ESC baseline on 8 of the 9 matrices, by  $1.27\times$  in geometric mean and by up to  $1.49\times$ , and outperforms MKLS on all 9, by  $1.55\times$  in geometric mean and by up to  $2.76\times$ . The margin over MKLS is widest on the two



**Figure 21: Multi-source BFS throughput on synthetic G500 graphs, where a frontier matrix holding  $\#source$  sources is repeatedly multiplied by the adjacency matrix. Subfigures (a)–(l) sweep the graph  $scale$  over 17–19 and the number of sources over  $2^{10}$ – $2^{16}$ , with  $edgefactor$  varying along the x-axis of every panel.**



**Figure 22: Triangle counting throughput on nine SuiteSparse matrices.**

most skewed of the nine, amazon-2008 and gsm\_106857 ( $2.76\times$  and  $1.72\times$ ), whereas the only case AMS-ESC loses is hugerick-00020, the least skewed matrix of the whole benchmark (Gini 0.0003), where there is hardly any imbalance to recover and the  $cf$ -based morsel sorting is not amortized. This result confirms the superior benefit of AMS under skew in real-world sorted graph regimes.

**4.7.4 Markov Clustering.** We finally evaluate AMS within an end-to-end graph-clustering pipeline, namely the HipMCL [7, 84] Markov clustering benchmark. HipMCL identifies clusters in protein-similarity

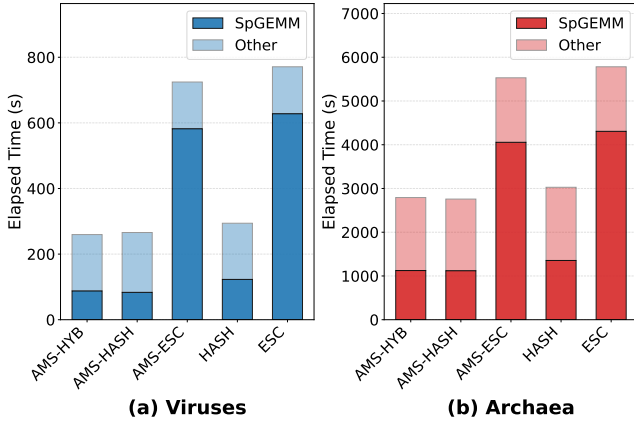
**Table 7: Summary of HipMCL Datasets: Viruses and Archaea [7, 84]**

| Dataset | $ vertices $ | $ edges $   | Gini   | Iters. to Converge <sup>*</sup> |
|---------|--------------|-------------|--------|---------------------------------|
| Viruses | 219,715      | 9,166,070   | 0.6574 | 37                              |
| Archaea | 1,644,227    | 409,568,764 | 0.5759 | 51                              |

<sup>\*</sup>Number of iterations until the algorithm converges to find clusters.

networks by simulating concurrent random walks over the graph, which are realized as repeated sparse matrix squaring of the similarity matrix until a stable clustering structure emerges. Across iterations, the similarity matrix becomes progressively sparser as the clustering structure crystallizes, yet sparse matrix squaring still dominates the overall runtime. We therefore integrate our SpGEMM implementations into HipMCL and report both the SpGEMM kernel time and the total end-to-end execution time on the two HipMCL datasets summarized in Table 7.

Figure 23 reports the results. AMS-HYB and AMS-HASH consistently deliver the best performance on both workloads, owing to their effective load balancing and on-chip cache contention alleviation. Compared with the HASH baseline, AMS-HASH reduces SpGEMM kernel runtime by 47.15% on Viruses and 20.91%



**Figure 23: Comparison of HipMCL [7] execution times for (a) Viruses and (b) Archaea datasets. Each bar is partitioned into the time spent on the specific SpGEMM kernel (dark) and the overhead from the remaining HipMCL framework (light).**

on Archaea, translating into 10.63% and 9.78% improvements in end-to-end HipMCL execution time, respectively. The larger gain on Viruses can be attributed to two factors: (1) Viruses exhibits a slightly more skewed similarity structure (Gini = 0.6574 vs. 0.5759), which amplifies the benefits of AMS’s skew-aware scheduling; and (2) Archaea requires substantially more MCL iterations to converge, and the later iterations operate on increasingly sparse matrices where load imbalance and cache contention become less pronounced, thus diluting AMS’s relative advantage. For ESC-based methods, the requirement of producing sorted output incurs noticeably higher SpGEMM execution time overall; nevertheless, AMS-ESC still outperforms the ESC baseline thanks to its superior load balancing and cache contention mitigation. Finally, AMS-HYB closely matches AMS-HASH on both workloads, confirming that HYB reliably selects the appropriate accumulator on a per-morsel basis even under the rapidly evolving sparsity patterns of MCL iterations.

## 4.8 Discussion

Our experimental analysis across sparse matrix multiplication, relational workloads, and graph analytics, on diverse datasets and hardware platforms, reveals several key findings that collectively validate the design rationale and practical effectiveness of AMS.

**Multi-dimensional skew is the primary performance bottleneck.** Across all matrix squaring evaluations (§ 4.2), we observe that the performance gap between AMS and flop-count-based baselines widens as workload skew increases. On non-skewed ER matrices, AMS performs comparably to the baselines, confirming that it introduces negligible overhead. On skewed G500 matrices, however, AMS delivers substantial gains (up to 81.01% for HASH), demonstrating that multi-dimensional skew, not algorithmic inefficiency, is the dominant factor limiting existing methods. That said, AMS may occasionally underperform in highly sparse cases due to overhead from *cf*-based morsel sorting; however, this penalty diminishes rapidly as matrix density increases, and is outweighed

by the substantial gains on the skewed workloads that dominate real-world applications.

### Contention-aware scheduling is essential, not optional.

The dissection of CALB and SMTact (§ 4.3) shows that simply achieving load balance is insufficient: methods that balance load but ignore cache contention (e.g., OpenMP dynamic and guided scheduling) still fall short of CALB. Similarly, SMTact’s selective SMT activation outperforms both always-on and always-off SMT configurations, confirming that the trade-off between computational throughput and cache pressure must be managed explicitly rather than left to a fixed policy.

**Load balancing unlocks hardware potential.** The CXL evaluation (§ 4.4) demonstrates that load imbalance can negate the benefits of bandwidth expansion, as straggler threads are disproportionately penalized by CXL’s higher access latency. By addressing this imbalance, AMS enables effective utilization of the augmented bandwidth, achieving throughput that exceeds the DRAM-only peak. This finding has broader implications. As heterogeneous memory systems become more prevalent, contention-aware load balancing will be increasingly critical for realizing their performance potential.

**The framework generalizes to other sparse kernels.** The SpMM and SpMV evaluation (§ 4.5) shows that AMS’s scheduling principles are not specific to SpGEMM but apply to sparse kernels with irregular sparsity in general. The bandwidth measurements expose the reason, as the baseline sustains only a fraction of the physical DRAM bandwidth on skewed workloads, whereas AMS recovers much of it as throughput. Since both kernels are memory-bound, contention-aware scheduling ultimately buys a better utilized memory system.

**SpGEMM-based methods are superior for relational query processing.** § 4.6 confirms the superiority of matrix-based methods on join-aggregate/project workloads, especially in the cases where the intermediate join result is large, from which the traditional join-then-aggregate/project methods suffer significantly, completing on only the smallest instances. In contrast, SpGEMM-based methods bypass that intermediate result entirely, saving its massive cost and thereby executing far faster. However, mapping such queries to sparse matrix multiplication algorithmically is not enough on its own, as the suboptimal performance of past matrix-based works [33, 54, 55, 102] demonstrates that efficient parallel execution is of utmost importance for reaching a satisfactory performance. AMS fills exactly this gap by pairing the remapping with contention-aware load balancing, and thereby turns an algorithmic advantage into a consistent runtime gain.

**The gains carry across graph analytics, from kernels to end-to-end pipelines.** The graph-analytic evaluation (§ 4.7) exercises AMS on four workloads that differ in operand shape, in output convention, and in scope, ranging from a single kernel to a complete pipeline. AMS leads the strongest baseline on PathSim meta-path construction, on non-sorted multi-source BFS, and on sorted triangle counting alike, so neither the tall-skinny nor the triangular operand, nor the requirement of row-wise ordered output, removes its advantage. On HipMCL, these kernel-level improvements translate into measurable end-to-end speedups in a real-world Markov clustering pipeline, even though the underlying matrices become progressively sparser across iterations. Across all four workloads,

the margin tracks the skew actually present, being largest on the most skewed inputs and smallest where the input carries virtually none, which reinforces that the practical value of contention-aware scheduling grows with workload skew.

## 5 RELATED WORK

SpGEMM has been extensively studied as a critical kernel in scientific computing and graph analytics [31, 87], and has more recently emerged in relational query processing [33, 54, 55, 59, 66, 102]. These relational works, however, are largely confined to the algorithmic mapping from queries to sparse matrix products, and address neither the efficient parallel execution of the resulting kernel nor the load imbalance under skew that AMS targets. The closest in spirit is SPALM [66], which groups the non-empty subrows of each operand into small slices and selects a multiplication kernel per slice pair from the local density, against thresholds calibrated once at installation for the host machine and data type. Its adaptivity therefore concerns which kernel runs on a given region, and it divides the parallel work so that threads write to disjoint locations, leaving load imbalance and cache contention out of scope, whereas AMS sizes and orders the parallel tasks themselves, so the two are complementary rather than competing.

Beneath every such mapping lies the SpGEMM kernel itself, whose optimization forms a second and far older line of work. Here Gustavson’s row-row formulation [50] has over the decades become the dominant approach due to its natural parallelizability, particularly when paired with the dense array accumulator known as SPA [44]. However, SPA has been shown to be inefficient and to incur high memory overhead, which prompted the development of more memory-efficient alternatives such as hash tables [84, 93], heaps [83], and radix sort [31].

In addition to algorithmic advances, much attention has been given to the challenges of load balancing in parallel SpGEMM. Schemes based on flop counts [83, 84] have emerged as a dominant solution, along with other static strategies such as methods using blocks [20, 21] or graph partitioning [2]. While effective in certain scenarios, these approaches often fail to account for the multi-dimensional skew characteristics present in real-world SpGEMM workloads, resulting in imbalanced execution.

As a more adaptive alternative, dynamic scheduling offers greater flexibility. For instance, Patwary et al. [88] proposed a dynamic task scheduling scheme that partitions work across both row and column dimensions. However, task partitioning across two dimensions introduces substantial synchronization overhead, which limits its practical applicability. Similarly, techniques leveraging work-stealing [16, 17, 125] and cache-access modeling [49] have been explored, but either incur significant cache coherence overhead or fail to account for complicated LLC sharing characteristics, making them less favorable than approaches based on flop counts. In another line of work, researchers have adapted dynamic variants of the classical longest-processing-time-first (LPT) algorithm [46], which prioritizes heavy tasks over lighter ones. Critically, none of these approaches account for on-chip cache contention caused by concurrently executing dense submatrices, which AMS explicitly addresses alongside multi-dimensional skew.

Prior work has likewise studied task granularity in SpGEMM, spanning one- and two-dimensional blocking [20, 21], sparse tiles [51, 86], and adaptive panels [124], each trading spatial locality against the freedom to balance work. The same trade-off arises in data systems, where morsel-driven parallelism [35, 72] schedules fixed-size fragments of *input* tuples across workers. AMS borrows the term but not the fixed input size, sizing each morsel adaptively to hold the *output* volume fixed. It further orders the dispatch to mitigate the contention that concurrently running morsels create in the shared cache, an objective that morsel-driven parallelism does not pursue. The two lines of work are therefore orthogonal, and the exchange can also run in the other direction, as a morsel-driven engine could order its dispatch and decide SMT activation from partition-size estimates when hash-based operators run over skewed keys.

## 6 CONCLUSION

This paper presents adaptive morsel scheduling (AMS), a morsel-wise dynamic scheduling framework that addresses two co-existing challenges in SpGEMM, namely *multi-dimensional skew* and *on-chip cache contention*. AMS balances skewed workloads without aggravating cache contention through contention-aware load balancing (CALB), judiciously activates SMT via data-conscious simultaneous multithreading activator (SMTact), and adaptively selects the per-morsel accumulator with hybrid accumulation mechanism (HYB). Evaluations across matrix squaring, relational join-aggregate and join-project queries, graph analytics, and other sparse kernels show that AMS significantly outperforms state-of-the-art methods on skewed workloads, maintains comparable performance on non-skewed ones, and improves bandwidth utilization, all of which hold on both conventional and CXL-expanded memory systems. Looking ahead, incorporating other SpGEMM formulations (e.g., outer-product or inner-product) into the morsel-wise scheduling framework presents a promising direction for further improvements, particularly on highly sparse matrices.

## REFERENCES

- [1] Christopher R. Aberger, Andrew Lamb, Kunle Olukotun, and Christopher Ré. 2018. LevelHeaded: A Unified Engine for Business Intelligence and Linear Algebra Querying. In *ICDE*. IEEE Computer Society, 449–460.
- [2] Kadir Akbudak and Cevdet Aykanat. 2014. Simultaneous Input and Output Matrix Partitioning for Outer-Product-Parallel Sparse Matrix-Matrix Multiplication. *SIAM J. Sci. Comput.* 36, 5 (2014).
- [3] Rasmus Resen Amossen and Rasmus Pagh. 2009. Faster join-projects and sparse matrix multiplications. In *Proceedings of the 12th International Conference on Database Theory*. 121–126.
- [4] Xiaojing An and Ümit V Çatalyürek. 2021. Column-segmented sparse matrix-matrix multiplication on multicore CPUs. In *2021 IEEE 28th International Conference on High Performance Computing, Data, and Analytics (HiPC)*. IEEE, 202–211.
- [5] Junya Arai, Hiroaki Shiokawa, Takeshi Yamamuro, Makoto Onizuka, and Sotetsu Iwamura. 2016. Rabbit Order: Just-in-Time Parallel Reordering for Fast Graph Analysis. In *IPDPS*. IEEE Computer Society, 22–31.
- [6] Ariful Azad, Grey Ballard, Aydın Buluç, James Demmel, Laura Grigori, Oded Schwartz, Sivan Toledo, and Samuel Williams. 2016. Exploiting Multiple Levels of Parallelism in Sparse Matrix-Matrix Multiplication. *SIAM J. Sci. Comput.* 38, 6 (2016).
- [7] Ariful Azad, Georgios A Pavlopoulos, Christos A Ouzounis, Nikos K Kyrpides, and Aydın Buluç. 2018. HipMCL: a high-performance parallel implementation of the Markov clustering algorithm for large-scale networks. *Nucleic acids research* 46, 6 (2018), e33–e33.
- [8] Cagri Ballesen, Gustavo Alonso, Jens Teubner, and M. Tamer Özsu. 2013. Multi-Core, Main-Memory Joins: Sort vs. Hash Revisited. *Proc. VLDB Endow.* 7, 1 (2013), 85–96.
- [9] Cagri Ballesen, Jens Teubner, Gustavo Alonso, and M. Tamer Özsu. 2013. Main-memory hash joins on multi-core CPUs: Tuning to the underlying hardware.

- In *ICDE*. IEEE Computer Society, 362–373.
- [10] Cagri Balkesen, Jens Teubner, Gustavo Alonso, and M. Tamer Özsu. 2015. Main-Memory Hash Joins on Modern Processor Architectures. *IEEE Trans. Knowl. Data Eng.* 27, 7 (2015), 1754–1766.
  - [11] Grey Ballard, Alex Druinsky, Nicholas Knight, and Oded Schwartz. 2016. Hypergraph Partitioning for Sparse Matrix-Matrix Multiplication. *ACM Trans. Parallel Comput.* 3, 3 (2016), 18:1–18:34.
  - [12] Maximilian Bandle, Jana Giceva, and Thomas Neumann. 2021. To Partition, or Not to Partition, That is the Join Question in a Real System. In *SIGMOD Conference*. ACM, 168–180.
  - [13] Richard Barrett, Michael Berry, Tony F Chan, James Demmel, June Donato, Jack Dongarra, Victor Eijkhout, Roldan Pozo, Charles Romine, and Henk Van der Vorst. 1994. *Templates for the solution of linear systems: building blocks for iterative methods*. SIAM.
  - [14] Nathan Bell, Steven Dalton, and Luke N Olson. 2012. Exposing fine-grained parallelism in algebraic multigrid methods. *SIAM Journal on Scientific Computing* 34, 4 (2012), C123–C152.
  - [15] Spyros Blanas, Yinan Li, and Jignesh M. Patel. 2011. Design and evaluation of main memory hash join algorithms for multi-core CPUs. In *SIGMOD Conference*. ACM, 37–48.
  - [16] Robert D. Blumofe. 1994. Scheduling Multithreaded Computations by Work Stealing. In *FOCS*. IEEE Computer Society, 356–368.
  - [17] Robert D. Blumofe, Christopher F. Joerg, Bradley C. Kuszmaul, Charles E. Leiserson, Keith H. Randall, and Yuli Zhou. 1995. Cilk: An Efficient Multithreaded Runtime System. In *PPoPP*. ACM, 207–216.
  - [18] OpenMP Architecture Review Board. 1997. OpenMP Fortran Application Program Interface. (1997).
  - [19] Peter A. Boncz, Stefan Manegold, and Martin L. Kersten. 1999. Database Architecture Optimized for the New Bottleneck: Memory Access. In *VLDB*. Morgan Kaufmann, 54–65.
  - [20] Aydin Buluç and John R. Gilbert. 2008. Challenges and Advances in Parallel Sparse Matrix-Matrix Multiplication. In *ICPP*. IEEE Computer Society, 503–510.
  - [21] Aydin Buluç and John R. Gilbert. 2008. On the representation and multiplication of hypersparse matrices. In *IPDPS*. IEEE, 1–11.
  - [22] Aydin Buluç and John R. Gilbert. 2011. The Combinatorial BLAS: design, implementation, and applications. *Int. J. High Perform. Comput. Appl.* 25, 4 (2011), 496–509.
  - [23] Aydin Buluç and John R. Gilbert. 2012. Parallel Sparse Matrix-Matrix Multiplication and Indexing: Implementation and Experiments. *SIAM J. Sci. Comput.* 34, 4 (2012).
  - [24] Deepayan Chakrabarti, Yiping Zhan, and Christos Faloutsos. 2004. R-MAT: A Recursive Model for Graph Mining. In *SDM*. SIAM, 442–446.
  - [25] Surajit Chaudhuri and Kyuseok Shim. 1994. Including Group-By in Query Optimization. In *VLDB '94, Proceedings of 20th International Conference on Very Large Data Bases, September 12-15, 1994, Santiago de Chile, Chile*, Jorge B. Bocca, Matthias Jarke, and Carlo Zaniolo (Eds.). Morgan Kaufmann, 354–366.
  - [26] YuAng Chen and Yeh-Ching Chung. 2022. Workload Balancing via Graph Reordering on Multicore Systems. *IEEE Trans. Parallel Distributed Syst.* 33, 5 (2022), 1231–1245.
  - [27] Yu Chen and Ke Yi. 2017. Two-Level Sampling for Join Size Estimation. In *SIGMOD Conference*. ACM, 759–774.
  - [28] Helin Cheng, Wenxuan Li, Yuechen Lu, and Weifeng Liu. 2023. HASpGEMM: Heterogeneity-Aware Sparse General Matrix-Matrix Multiplication on Modern Asymmetric Multicore Processors. In *ICPP*. ACM, 807–817.
  - [29] Transaction Processing Performance Council. 2021. TPC-BENCHMARKDM DS Standard Specification Revision 3.2.0. [https://www.tpc.org/tpc\\_documents\\_current\\_versions/pdf/tpc-ds\\_v3.2.0.pdf](https://www.tpc.org/tpc_documents_current_versions/pdf/tpc-ds_v3.2.0.pdf)
  - [30] Leonardo Dagum and Ramesh Menon. 1998. OpenMP: an industry standard API for shared-memory programming. *IEEE computational science and engineering* 5, 1 (1998), 46–55.
  - [31] Steven Dalton, Luke Olson, and Nathan Bell. 2015. Optimizing sparse matrix-matrix multiplication for the gpu. *ACM Transactions on Mathematical Software (TOMS)* 41, 4 (2015), 1–20.
  - [32] Timothy A. Davis and Yifan Hu. 2011. The university of Florida sparse matrix collection. *ACM Trans. Math. Softw.* 38, 1 (2011), 1:1–1:25.
  - [33] Shaleen Deep, Xiao Hu, and Paraschos Koutiris. 2020. Fast join project query evaluation using matrix multiplication. In *Proceedings of the 2020 ACM SIGMOD International Conference on Management of Data*. 1213–1223.
  - [34] Laxman Dhulipala, Charles McGuffey, Hongbo Kang, Yan Gu, Guy E. Blelloch, Phillip B. Gibbons, and Julian Shun. 2020. Sage: Parallel Semi-Asymmetric Graph Algorithms for NVRAMs. *Proc. VLDB Endow.* 13, 9 (2020), 1598–1613.
  - [35] Kayhan Dursun, Carsten Binnig, Ugur Cetintemel, Garret Swart, and Weiwei Gong. 2019. A Morsel-Driven Query Execution Engine for Heterogeneous Multi-Cores. *Proc. VLDB Endow.* 12, 12 (2019), 2218–2229.
  - [36] Marius Eich, Pit Fender, and Guido Moerkotte. 2018. Efficient generation of query plans containing group-by, join, and groupjoin. *VLDB J.* 27, 5 (2018), 617–641.
  - [37] Philipp Fent, Altan Birler, and Thomas Neumann. 2023. Practical planning and execution of groupjoin and nested aggregates. *VLDB J.* 32, 6 (2023), 1165–1190.
  - [38] Philipp Fent and Thomas Neumann. 2021. A Practical Approach to Groupjoin and Nested Aggregates. *Proc. VLDB Endow.* 14, 11 (2021), 2383–2396.
  - [39] Patrick J. Flynn, Xinyao Yi, Erik Saule, Gokcen Kestor, and Yonghong Yan. 2025. SpMM-Bench: Performance Characterization of Sparse Formats for Sparse-Dense Matrix Multiplication. In *IPDPS (Workshops)*. IEEE, 149–158.
  - [40] Jianhua Gao, Weixing Ji, Fangli Chang, Shiyu Han, Bingxin Wei, Zeming Liu, and Yizhuo Wang. 2023. A Systematic Survey of General Sparse Matrix-matrix Multiplication. *ACM Comput. Surv.* 55, 12 (2023), 244:1–244:36.
  - [41] Jianhua Gao, Bingjie Liu, Weixing Ji, and Hua Huang. 2024. A Systematic Literature Survey of Sparse Matrix-Vector Multiplication. *CoRR abs/2404.06047* (2024).
  - [42] Sen Gao, Shengliang Lu, Shixuan Sun, Yuchen Li, and Bingsheng He. 2025. An Efficient Memoization Engine for Concurrent Graph Query Processing. In *ICDE*. IEEE, 1015–1028.
  - [43] Amir Gholami, Zhewei Yao, Sehoon Kim, Coleman Hooper, Michael W. Mahoney, and Kurt Keutzer. 2024. AI and Memory Wall. *IEEE Micro* 44, 3 (2024), 33–39.
  - [44] John R. Gilbert, Cleve Moler, and Robert Schreiber. 1992. Sparse matrices in MATLAB: Design and implementation. *SIAM journal on matrix analysis and applications* 13, 1 (1992), 333–356.
  - [45] John R. Gilbert, Steven P. Reinhardt, and Viral B. Shah. 2006. High-Performance Graph Algorithms from Parallel Sparse Matrices. In *PARA (Lecture Notes in Computer Science)*, Vol. 4699. Springer, 260–269.
  - [46] Ronald L. Graham. 1969. Bounds on Multiprocessing Timing Anomalies. *SIAM Journal of Applied Mathematics* 17, 2 (1969), 416–429.
  - [47] Zhixiang Gu, Jose Moreira, David Edelsohn, and Ariful Azad. 2020. Bandwidth Optimized Parallel Algorithms for Sparse Matrix-Matrix Multiplication using Propagation Blocking. In *SPAA*. ACM, 293–303.
  - [48] Gaël Guennebaud, Benoît Jacob, et al. 2010. Eigen. <https://libeigen.gitlab.io/>
  - [49] Jihu Guo, Rui Xia, Jie Liu, Xiaoxiong Zhu, and Xiang Zhang. 2024. CAMLB-SpMV: An Efficient Cache-Aware Memory Load-Balancing SpMV on CPU. In *ICPP*. ACM, 640–649.
  - [50] Fred G. Gustavson. 1978. Two Fast Algorithms for Sparse Matrices: Multiplication and Permuted Transposition. *ACM Trans. Math. Softw.* 4, 3 (1978), 250–269.
  - [51] Changwan Hong, Aravind Sukumaran-Rajam, Israt Nisa, Kunal Singh, and P. Sadayappan. 2019. Adaptive sparse tiling for sparse matrix multiplication. In *PPoPP*. ACM, 300–314.
  - [52] Chuhe Hong, Qinglin Wang, Runzhang Mao, Yuechao Liang, Rui Xia, and Jie Liu. 2024. SaSpGEMM: Sorting-Avoiding Sparse General Matrix-Matrix Multiplication on Multi-Core Processors. In *ICPP*. ACM, 1166–1175.
  - [53] Yuxi Hong and Aydin Buluç. 2024. A Sparsity-Aware Distributed-Memory Algorithm for Sparse-Sparse Matrix Multiplication. In *SC*. IEEE, 47.
  - [54] Xiao Hu. 2025. Output-optimal algorithms for join-aggregate queries. *Proceedings of the ACM on Management of Data* 3, 2 (2025), 1–27.
  - [55] Xiao Hu and Ke Yi. 2020. Parallel algorithms for sparse matrix multiplication and join-aggregate queries. In *Proceedings of the 39th ACM SIGMOD-SIGACT-SIGAI Symposium on Principles of Database Systems*. 411–425.
  - [56] Yu-Ching Hu, Yuliang Li, and Hung-Wei Tseng. 2022. TCUDB: Accelerating Database with Tensor Processors. In *SIGMOD Conference*. ACM, 1360–1374.
  - [57] Wentao Huang, Mian Lu, and Kian-Lee Tan. 2026. Hash Joins Meet CXL: A Fresh Look. In *CIDR*. [www.cidrdb.org](http://www.cidrdb.org).
  - [58] Wentao Huang, Mo Sha, Mian Lu, Yuqiang Chen, Bingsheng He, and Kian-Lee Tan. 2024. Bandwidth Expansion via CXL: A Pathway to Accelerating In-Memory Analytical Processing. In *VLDB Workshops*. VLDB.org.
  - [59] Zichun Huang and Shimin Chen. 2022. Density-optimized intersection-free mapping and matrix multiplication for join-project operations. *Proceedings of the VLDB Endowment* 15, 10 (2022), 2244–2256.
  - [60] IMDb.com, Inc. 2024. IMDb Non-Commercial Datasets. <https://developer.imdb.com/non-commercial-datasets/>
  - [61] Intel. 2024. Intel® oneAPI Math Kernel Library (oneMKL). <https://www.intel.com/content/www/us/en/developer/tools/oneapi/onemkl.html>
  - [62] Myung-Hwan Jang, Yunyong Ko, Hyuck-Moo Gwon, Ikhyeon Jo, Yongjun Park, and Sang-Wook Kim. 2023. SAGE: a storage-based approach for scalable and efficient sparse generalized matrix-matrix multiplication. In *Proceedings of the 32nd ACM International Conference on Information and Knowledge Management*. 923–933.
  - [63] Mahmoud Abo Khamis, Xiao Hu, and Dan Suciu. 2025. Fast Matrix Multiplication meets the Submodular Width. *Proc. ACM Manag. Data* 3, 2 (2025), 98:1–98:26.
  - [64] Farzad Khorasani, Rajiv Gupta, and Laxmi N. Bhuyan. 2015. Scalable SIMD-Efficient Graph Processing on GPUs. In *Proceedings of the 24th International Conference on Parallel Architectures and Compilation Techniques (PACT '15)*. 39–50.
  - [65] Changkyu Kim, Eric Sedlar, Jatin Chhugani, Tim Kaldewey, Anthony D. Nguyen, Andrea Di Blas, Victor W. Lee, Nadathur Satish, and Pradeep Dubey. 2009. Sort

- vs. Hash Revisited: Fast Join Implementation on Modern Multi-Core CPUs. *Proc. VLDB Endow.* 2, 2 (2009), 1378–1389.
- [66] Junyoung Kim and Kenneth A. Ross. 2026. SPALM: A Sparsity-Pattern-Adaptive Library for Matrices. *Proc. ACM Manag. Data* 4, 3 (2026), 236:1–236:27.
- [67] Denis Kotkov, Joseph A. Konstan, Qian Zhao, and Jari Veijalainen. 2018. Investigating serendipity in recommender systems based on real user feedback. In *SAC*. ACM, 1341–1350.
- [68] Denis Kotkov, Alan Medlar, Alexandr Maslov, Umesh Raj Satyal, Mats Neovius, and Dorota Glowacka. 2022. The Tag Genome Dataset for Books. In *Proceedings of the 2022 ACM SIGIR Conference on Human Information Interaction and Retrieval (CHIIR '22)*. 5. <https://doi.org/10.1145/3498366.3505833>
- [69] Jérôme Kunegis and Julia Preusse. 2012. Fairness on the web: alternatives to the power law. In *WebSci*. ACM, 175–184.
- [70] Georgiy Lebedev, Hamish Nicholson, Musa Ünäl, Sanidhya Kashyap, and Anastasia Ailamaki. 2025. Demystifying CXL Memory Bandwidth Expansion for Analytical Workloads. In *VLDB Workshops*. VLDB.org.
- [71] Suhyun Lee, Chaemin Lim, Jinwoo Choi, Heelim Choi, Chan Lee, Yongjun Park, Kwanghyun Park, Hanjun Kim, and Youngsok Kim. 2024. SPID-Join: A Skew-resistant Processing-in-DIMM Join Algorithm Exploiting the Bank- and Rank-level Parallelisms of DIMMs. *Proc. ACM Manag. Data* 2, 6 (2024), 251:1–251:27.
- [72] Viktor Leis, Peter A. Boncz, Alfons Kemper, and Thomas Neumann. 2014. Morsel-driven parallelism: a NUMA-aware query evaluation framework for the many-core age. In *SIGMOD Conference*. ACM, 743–754.
- [73] Alberto Lerner and Gustavo Alonso. 2024. CXL and the Return of Scale-Up Database Engines. *Proc. VLDB Endow.* 17, 10 (2024), 2568–2575.
- [74] Jure Leskovec, Deepayan Chakrabarti, Jon Kleinberg, Christos Faloutsos, and Zoubin Ghahramani. 2010. Kronecker graphs: an approach to modeling networks. *Journal of Machine Learning Research* 11, 2 (2010).
- [75] Zhonggen Li, Xiangyu Ke, Yifan Zhu, Yunjun Gao, and Yaofeng Tu. 2025. HC-SpMM: Accelerating Sparse Matrix-Matrix Multiplication for Graphs with Hybrid GPU Cores. In *ICDE*. IEEE, 501–514.
- [76] Chunxu Lin, Wensheng Luo, Yixiang Fang, Chenhao Ma, Xilin Liu, and Yuchi Ma. 2024. On Efficient Large Sparse Matrix Chain Multiplication. *Proc. ACM Manag. Data* 2, 3 (2024), 156.
- [77] Jinshu Liu, Hamid Hadian, Yuyue Wang, Daniel S. Berger, Marie Nguyen, Xun Jian, Sam H. Noh, and Huaicheng Li. 2025. Systematic CXL Memory Characterization and Performance Analysis at Scale. In *ASPLOS (2)*. ACM, 1203–1217.
- [78] Jinshu Liu, Hanchen Xu, Daniel S. Berger, Marcos K. Aguilera, and Huaicheng Li. 2026. Performance Predictability in Heterogeneous Memory. In *ASPLOS (2)*. ACM, 1413–1429.
- [79] Shangyu Luo, Zekai J. Gao, Michael N. Gubanov, Luis Leopoldo Perez, and Christopher M. Jermaine. 2017. Scalable Linear Algebra on a Relational Database System. In *ICDE*. IEEE Computer Society, 523–534.
- [80] Stefan Manegold, Peter A. Boncz, and Martin L. Kersten. 2002. Optimizing Main-Memory Join on Modern Hardware. *IEEE Trans. Knowl. Data Eng.* 14, 4 (2002), 709–730.
- [81] Guido Moerkotte and Thomas Neumann. 2011. Accelerating Queries with Group-By and Join by Groupjoin. *Proc. VLDB Endow.* 4, 11 (2011), 843–851.
- [82] Ingo Müller, Peter Sanders, Arnaud Lacurie, Wolfgang Lehner, and Franz Färber. 2015. Cache-Efficient Aggregation: Hashing Is Sorting. In *SIGMOD Conference*. ACM, 1123–1136.
- [83] Yusuke Nagasaka, Satoshi Matsuoka, Ariful Azad, and Aydin Buluç. 2018. High-Performance Sparse Matrix-Matrix Products on Intel KNL and Multicore Architectures. In *ICPP Workshops*. ACM, 34:1–34:10.
- [84] Yusuke Nagasaka, Satoshi Matsuoka, Ariful Azad, and Aydin Buluç. 2019. Performance optimization, modeling and analysis of sparse matrix-matrix products on multi-core and many-core processors. *Parallel Comput.* 90 (2019).
- [85] Tien T. Nguyen, F. Maxwell Harper, Loren Terveen, and Joseph A. Konstan. 2018. User Personality and User Satisfaction with Recommender Systems. *Inf. Syst. Frontiers* 20, 6 (2018), 1173–1189.
- [86] Yuyao Niu, Zhengyang Lu, Haonan Ji, Shuhui Song, Zhou Jin, and Weifeng Liu. 2022. TileSpGEMM: a tiled algorithm for parallel sparse general matrix-matrix multiplication on GPUs. In *PPoPP*. ACM, 90–106.
- [87] Taehyeong Park, Seokwon Kang, Myung-Hwan Jang, Sang-Wook Kim, and Yongjun Park. 2023. Orchestrating Large-Scale SpGEMMs using Dynamic Block Distribution and Data Transfer Minimization on Heterogeneous Systems. In *ICDE*. IEEE, 2456–2459.
- [88] Md. Mostofa Ali Patwary, Nadathur Rajagopalan Satish, Narayanan Sundaram, Jongsoo Park, Michael J. Anderson, Satya Gautam Vadlamudi, Dipankar Das, Sergey G. Pudov, Vadim O. Pirogov, and Pradeep Dubey. 2015. Parallel Efficient Sparse Matrix-Matrix Multiplication on Multicore Platforms. In *ISC (Lecture Notes in Computer Science)*, Vol. 9137. Springer, 48–57.
- [89] Constantine D. Polychronopoulos and David J. Kuck. 1987. Guided Self-Scheduling: A Practical Scheduling Scheme for Parallel Supercomputers. *IEEE Trans. Computers* 36, 12 (1987), 1425–1439.
- [90] Eric Qin, Ananda Samajdar, Hyoukjun Kwon, Vineet Nadella, Sudarshan Srinivasan, Dipankar Das, Bharat Kaul, and Tushar Krishna. 2020. SIGMA: A Sparse and Irregular GEMM Accelerator with Flexible Interconnects for DNN Training. In *2020 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. 58–70. <https://doi.org/10.1109/HPCA47549.2020.00015>
- [91] Moinuddin K. Qureshi. 2014. Memory Scaling is Dead, Long Live Memory Scaling. [https://hps.ece.utexas.edu/yale75/qureshi\\_slides.pdf](https://hps.ece.utexas.edu/yale75/qureshi_slides.pdf)
- [92] Isuru Ranawaka, Md Taufique Hussain, Charles Block, Gerasimos Gerogiannis, Josep Torrellas, and Ariful Azad. 2024. Distributed-Memory Parallel Algorithms for Sparse Matrix and Sparse Tall-and-Skinny Matrix Multiplication. In *SC*. IEEE, 46.
- [93] Stefan Richter, Victor Alvarez, and Jens Dittrich. 2015. A seven-dimensional analysis of hashing methods and its implications on query processing. *Proceedings of the VLDB Endowment* 9, 3 (2015), 96–107.
- [94] Yousef Saad. 2003. *Iterative methods for sparse linear systems*. SIAM.
- [95] Tobias Schmidt, Viktor Leis, Peter Boncz, and Thomas Neumann. 2025. SQL-Storm: Taking Database Benchmarking into the LLM Era. *Proc. VLDB Endow.* 18, 11 (2025), 4144–4157.
- [96] Stefan Schuh, Xiao Chen, and Jens Dittrich. 2016. An Experimental Comparison of Thirteen Relational Equi-Joins in Main Memory. In *SIGMOD Conference*. ACM, 1961–1976.
- [97] Robert Schulze, Tom Schreiber, Ilya Yatsishin, Ryadh Dahimene, and Alexey Milovidov. 2024. ClickHouse - Lightning Fast Analytics for Everyone. *Proc. VLDB Endow.* 17, 12 (2024), 3731–3744.
- [98] Malte Skarupke. 2017. I Wrote The Fastest Hashtable. Probably Dance. <https://probablydance.com/2017/02/26/i-wrote-the-fastest-hashtable/> Accessed: 2025-02-21.
- [99] SuiteSparse Matrix Collection. 2018. VLSI/vas\_stokes\_1M. [https://sparse.tamu.edu/VLSI/vas\\_stokes\\_1M](https://sparse.tamu.edu/VLSI/vas_stokes_1M)
- [100] Ruixuan Sun, Ruoyan Kong, Qiao Jin, and Joseph A. Konstan. 2023. Less Can Be More: Exploring Population Rating Dispositions with Partitioned Models in Recommender Systems. In *UMAP (Adjunct Publication)*. ACM, 291–295.
- [101] Wenbo Sun, Asterios Katsifodimos, and Rihan Hai. 2023. Accelerating Machine Learning Queries with Linear Algebra Query Processing. In *SSDBM*. ACM, 13:1–13:12.
- [102] Wenbo Sun, Asterios Katsifodimos, and Rihan Hai. 2023. An Empirical Performance Comparison between Matrix Multiplication Join and Hash Join on GPUs. In *ICDEW*. IEEE, 184–190.
- [103] Wenbo Sun, Asterios Katsifodimos, and Rihan Hai. 2025. Accelerating machine learning queries with linear algebra query processing. *Distributed Parallel Databases* 43, 1 (2025), 8.
- [104] Yizhou Sun, Jiawei Han, Xifeng Yan, Philip S. Yu, and Tianyi Wu. 2011. PathSim: Meta Path-Based Top-K Similarity Search in Heterogeneous Information Networks. *Proc. VLDB Endow.* 4, 11 (2011), 992–1003.
- [105] Yan Sun, Yifan Yuan, Zeduo Yu, Reese Kuper, Chihun Song, Jinghan Huang, Houxiang Ji, Siddharth Agarwal, Jiaqi Lou, Ipoom Jeong, Ren Wang, Jung Ho Ahn, Tianyin Xu, and Nam Sung Kim. 2023. Demystifying CXL Memory with Genuine CXL-Ready Systems and Devices. In *MICRO*. ACM, 105–121.
- [106] Narayanan Sundaram, Nadathur Satish, Md. Mostofa Ali Patwary, Subramanya Dulloor, Michael J. Anderson, Satya Gautam Vadlamudi, Dipankar Das, and Pradeep Dubey. 2015. GraphMat: High performance graph analytics made productive. *Proc. VLDB Endow.* 8, 11 (2015), 1214–1225.
- [107] Jie Tang, Jing Zhang, Limin Yao, Juanzi Li, Li Zhang, and Zhong Su. 2017. DBLP-Citation-Network V10. AMiner Citation Network Dataset, release 10. <https://lfs.aminer.cn/lab-datasets/citation/dblp.v10.zip> Landing page: <https://www.aminer.org/citation.1.85GBarchive,4JSONshards,3,079,007papers>.
- [108] Yupeng Tang, Ping Zhou, Wenhui Zhang, Henry Hu, Qirui Yang, Hao Xiang, Tongping Liu, Jiaxin Shan, Ruoyun Huang, Cheng Zhao, Cheng Chen, Hui Zhang, Fei Liu, Shuai Zhang, Xiaoning Ding, and Jianjun Chen. 2024. Exploring Performance and Cost Optimization with ASIC-Based CXL Memory. In *EuroSys*. ACM, 818–833.
- [109] Micron Technology. 2023. Memory Scaling is Dead, Long Live Memory Scaling. <https://sg.micron.com/content/dam/micron/global/public/products/white-paper/cxl-impact-dram-bit-growth-white-paper.pdf>
- [110] The Linux Kernel Development Community. 2024. *Intel(R) Speed Select Technology User Guide*. <https://docs.kernel.org/admin-guide/pm/intel-speed-select.html> Linux Kernel Documentation.
- [111] James D. Trotter, Sinan Ekmekci, Johannes Langguth, Tugba Torun, Emre Düzakin, Aleksandar Ilic, and Didem Unat. 2023. Bringing Order to Sparsity: A Sparse Matrix Reordering Study on Multicore CPUs. In *SC*. ACM, 31:1–31:13.
- [112] Pauli Virtanen, Ralf Gommers, Travis E. Oliphant, et al. 2020. SciPy 1.0: Fundamental Algorithms for Scientific Computing in Python. *Nature Methods* 17, 3 (2020), 261–272. <https://doi.org/10.1038/s41592-019-0686-2>
- [113] Aleksandar Vitorovic, Mohammed Elseidy, and Christoph Koch. 2016. Load balancing and skew resilience for parallel joins. In *ICDE*. IEEE Computer Society, 313–324.
- [114] Jeffrey Scott Vitter. 1985. Random Sampling with a Reservoir. *ACM Trans. Math. Softw.* 11, 1 (1985), 37–57.
- [115] TaiNing Wang and Chee-Yong Chan. 2018. Improving Join Reorderability with Compensation Operators. In *SIGMOD Conference*. ACM, 693–708.

- [116] TaiNing Wang and Chee-Yong Chan. 2020. Improved Correlated Sampling for Join Size Estimation. In *ICDE*. IEEE, 325–336.
- [117] Marcel Weisgut, Daniel Ritter, Florian Schmeller, Pinar Tözün, and Tilmann Rabl. 2025. CXL-Bench: Benchmarking Shared CXL Memory Access. In *VLDB Workshops*. VLDB.org.
- [118] Marcel Weisgut, Daniel Ritter, Pinar Tözün, Lawrence Benson, and Tilmann Rabl. 2025. CXL Memory Performance for In-Memory Data Processing. *Proc. VLDB Endow.* 18, 9 (2025), 3119–3133.
- [119] Samuel Williams. 2008. Williams/webbase-1M. <https://sparse.tamu.edu/Williams/webbase-1M>
- [120] Martin Winter, Daniel Mlakar, Rhaleb Zayer, Hans-Peter Seidel, and Markus Steinberger. 2019. Adaptive sparse matrix-matrix multiplication on the GPU. In *PPoPP*. ACM, 68–81.
- [121] Guoqing Xiao, Chuanghui Yin, Tao Zhou, Xueqi Li, Yuedan Chen, and Kenli Li. 2024. A Survey of Accelerating Parallel Sparse Linear Algebra. *ACM Comput. Surv.* 56, 1 (2024), 21:1–21:38.
- [122] Weipeng P. Yan and Per-Ake Larson. 1994. Performing Group-By before Join. In *Proceedings of the Tenth International Conference on Data Engineering, February 14-18, 1994, Houston, Texas, USA*. IEEE Computer Society, 89–100.
- [123] Weipeng P. Yan and Per-Ake Larson. 1995. Eager Aggregation and Lazy Aggregation. In *VLDB*. Morgan Kaufmann, 345–357.
- [124] Dezhong Yao, Sifan Zhao, Tongtong Liu, Gang Wu, and Hai Jin. 2024. Ap-SpGEMM: Accelerating Large-scale SpGEMM with Heterogeneous Collaboration and Adaptive Panel. *ACM Transactions on Architecture and Code Optimization* (2024).
- [125] Abdurrahman Yaşar, Sivasankaran Rajamanickam, Michael Wolf, Jonathan Berry, and Ümit V Çatalyürek. 2018. Fast triangle counting using cilk. In *2018 IEEE High Performance Extreme Computing Conference (HPEC)*. IEEE, 1–7.
- [126] Mustafa Yazici. 2023. goodbooks-10k-the-latest. <https://www.kaggle.com/datasets/mustafayazici/goodbooks-10k-the-latest>. Accessed: 2026-02-21.
- [127] Serif Yesil, Azin Heidarshenas, Adam Morrison, and Josep Torrellas. 2020. Speeding up SpMV for power-law graph analytics by enhancing locality & vectorization. In *SC*. IEEE/ACM, 86.
- [128] Dongxiang Zhang, Dongsheng Li, Long Guo, and Kian-Lee Tan. 2022. Unsupervised Entity Resolution With Blocking and Graph Algorithms. *IEEE Trans. Knowl. Data Eng.* 34, 3 (2022), 1501–1515.

## APPENDIX

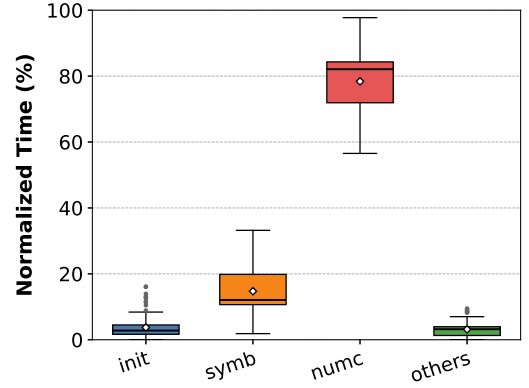
### A SPGEMM SYMBOLIC PHASE ANALYSIS

AMS targets the numeric phase of SpGEMM and executes the symbolic phase with standard dynamic scheduling over morsels of a fixed row count (§ 3), rather than over the output-sized morsels it forms for the numeric phase (§ 3.1). This section explains that choice with two arguments. First, the symbolic phase takes only a minor share of the end-to-end runtime, so optimizing it offers limited headroom. Second, even within that headroom, extending AMS to the symbolic phase yields marginal benefits at best, since the contention that AMS is designed to remove barely arises there.

We first make the current scheduling concrete. AMS cuts the rows of matrix  $A$  into morsels of a predefined row count and hands them to the threads from a shared queue, so that a thread claims the next morsel as soon as it finishes the current one. Grouping consecutive rows preserves the spatial locality of sparse matrices (§ 2.3), while dynamic dispatch keeps every thread busy even when a few rows are far heavier than the rest, so this scheme alone is already sufficient to balance the symbolic phase under skew. Notably, it involves neither a per-morsel cost estimate nor a prescribed dispatch order, and therefore carries no preparation of its own. The per-row counts it produces then drive the prefix sum and the output allocation, and they are what AMS consumes to build the output-sized morsels of the numeric phase (§ 3.1).

#### A.1 The Symbolic Phase Is a Minor Cost

Recall that a Gustavson-style SpGEMM proceeds in two phases [50, 84]. The symbolic phase merely counts the distinct column indices



**Figure 24: Distribution of the per-phase share of the end-to-end SpGEMM time of AMS-HYB over the 119 SuiteSparse matrices of § 4.1, computed per matrix. symb and numc denote the symbolic and numeric phases, init denotes AMS preparation (morsel construction, cf computation, and morsel sorting), and others denotes prefix sum, output allocation, and finalization.**

of each output row, whereas the numeric phase carries out the multiply-accumulate work and materializes the result. The symbolic phase therefore performs no multiplication, touches no numerical value, and materializes no output beyond a single per-row counter. It consequently moves far less data and suffers far less write-back and cache contention than the numeric phase, which already suggests that it occupies only a limited share of the end-to-end runtime.

Figure 24 confirms this expectation on real-world data, reporting the phase composition of AMS-HYB over the 119 SuiteSparse matrices [32] described in § 4.1. The symbolic phase accounts for an average of 12.07% of the end-to-end runtime, whereas the numeric phase accounts for an average of 82.09%, with AMS preparation (init) and the remaining management steps (others) contributing only a few percent each<sup>22</sup>. The numeric phase thus dominates end-to-end execution on real-world matrices, which is precisely where we concentrate our optimizations.

This composition holds regardless of the scheduling scheme applied to the symbolic phase itself. Figure 25 reports the same breakdown for the matched algorithms HASH and AMS-HASH on a non-skewed ER workload and on a skewed G500 workload, both at scale 19 and edgfactor 16. The two run an identical symbolic kernel and differ only in its scheduling scheme, since HASH partitions the rows statically by flop count [83, 84] whereas AMS-HASH dispatches morsels of a fixed row count dynamically, so the two cover a static and a dynamic schedule of the same work. Under both schedules and on both workloads, the symbolic phase accounts for no more than 20% of the end-to-end runtime. The effect is further amplified on the skewed workload, where the numeric phase claims an even larger proportion of the runtime and leaves the symbolic phase an even smaller proportional cost.

<sup>22</sup>We acknowledge that the runtime breakdown may exhibit a different percentage profile on another platform, but the overall trend and the relative order of the phases should remain coherent.

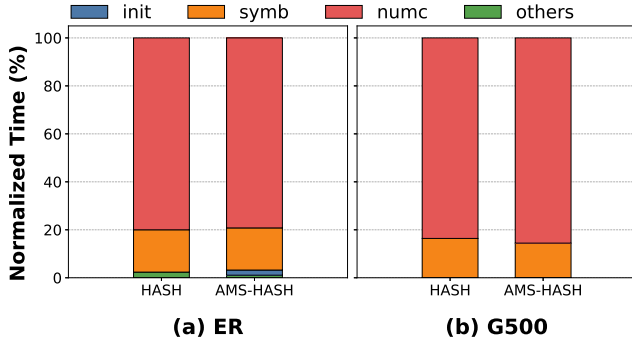


Figure 25: Normalized end-to-end runtime breakdown on ER and G500 at scale 19, edgfactor 16, where every bar is normalized to its own end-to-end SpGEMM time. Each bar is partitioned into the symbolic phase (symb), the numeric phase (numc), AMS’s morsel preparation (init), and the remaining management steps of prefix sum, output allocation, and finalization (others).

## A.2 Marginal Benefits in Extending AMS to the Symbolic Phase

We now examine whether the remaining headroom is worth claiming, i.e., whether extending AMS to the symbolic phase yields additional benefit. Doing so is mechanically straightforward. The whole AMS machinery carries over to the symbolic phase once we replace its output measure by an input measure, since this phase reads the nonzeros of matrix  $A$  but has not yet computed those of matrix  $C$ . Concretely, we grow a morsel over consecutive rows of matrix  $A$  until it reaches a target number of input nonzeros, and we locate the morsel boundaries by a binary search over the prefix sum of the per-row nonzero counts. We then give each morsel the ratio of its flop count to its input nonzero count, which plays here the role that  $cf$  plays in the numeric phase, and we sort the morsels by this ratio before CALB dispatches them. The procedure is thus the one of § 3.1 and § 3.2, with  $nnz(A)$  taking the place of  $nnz(C)$ . We denote the resulting variant AMS-HASH\*.

From the mechanism point of view, however, the expected benefit is small. CALB and SMTact are designed to relieve on-chip cache contention (§ 3.2 and § 3.3), yet the symbolic phase incurs far less write-back and contention than the numeric phase in the first place, so it offers little for the two techniques to remove. The input-side ratio is also a coarser signal than  $cf$ , because the symbolic phase materializes no output against which a morsel footprint could be budgeted, so a morsel of a fixed input size still leaves the accumulator footprint uncontrolled.

Figure 26 confirms this reasoning by reporting absolute runtimes for AMS-HASH\* on the same two workloads as Figure 25. On the skewed G500 workload, AMS-HASH reduces the end-to-end time from 35.2 s for HASH to 29.4 s, whereas AMS-HASH\* reaches only 29.1 s, a further gain of about 1%. The scheduling does shorten the symbolic phase itself from 4.8 s to 4.5 s, but the extra preparation it requires consumes most of that saving. On the non-skewed ER workload, the variant is in fact marginally slower (0.531 s against

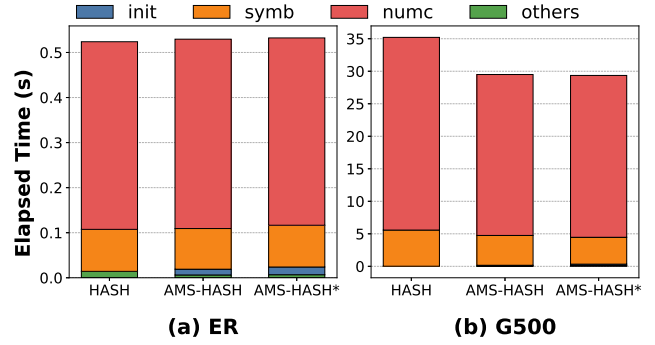


Figure 26: End-to-end runtime breakdown on ER and G500 at scale 19, edgfactor 16. AMS-HASH\* is a variant of AMS-HASH that additionally schedules the symbolic phase with CALB, instead of the standard dynamic scheduling with morsels of a fixed row count used by AMS-HASH. Note the different y-axis ranges of (a) and (b).

0.528 s), since there is little imbalance to recover and the added preparation is pure overhead.

This preparation overhead is not incidental, but inherent in the way AMS prepares its morsels. The variant has to compute the prefix sum over the per-row nonzero counts, build the morsels, evaluate their cost, and sort them, all before a single row is counted, whereas the current AMS scheme performs none of these steps. It is moreover charged most heavily to non-skewed workloads, which have no imbalance to recover in return, and doing so would violate our design principle of being **non-disruptive to non-skewed workloads** (§ 3). We therefore scope AMS to the numeric phase, and note that the symbolic phase calls for a different line of optimization than the contention-aware scheduling developed in this work.

## B OVERSIZED OUTPUT ROWS

In this section, we discuss the performance impact of oversized output rows, namely those rows whose own output already exceeds our predetermined morsel size, if such rows exist in a workload. Recall that AMS forms morsels against a target output footprint, which we set to half of the L2 cache capacity (§ 3.1). A morsel that holds an oversized row therefore cannot respect that budget, and may not even fit into the L2 cache. We study these rows over the full R-MAT [24] sweep, and reach three answers. Oversized rows are the common case rather than a corner case, crossing a cache boundary does not degrade AMS, and the width of the widest row is not the variable that governs the speedup of AMS.

### B.1 Oversized Rows Are the Common Case

Recall from § 3.1 that the morsel size is only a soft target, and it is so precisely because Gustavson’s algorithm forms the output row by row [50], which makes an output row atomic and leaves AMS no way to split one across threads. A morsel therefore grows by whole rows until it reaches the target, and a row whose own output already exceeds the target ends up as a morsel by itself.

**Table 8: Number of evaluated configurations whose widest output row exceeds a given capacity. The non-skewed ER covers *scale* 18–21, while SSCA and G500 cover *scale* 17–20, all at *edgefactor* 2–32.**

| widest output row of $C$ exceeds        | ER   | SSCA <sup>†</sup> | G500 <sup>†</sup> |
|-----------------------------------------|------|-------------------|-------------------|
| the morsel target, i.e., half of the L2 | 0/20 | 16/19             | 17/19             |
| the full L2 capacity                    | 0/20 | 11/19             | 12/19             |

<sup>†</sup>SSCA and G500 report 19 of their 20 grid points, as their largest setting, namely *scale* 20 with *edgefactor* 32, runs out of memory on our platform.

We now examine how often such oversized rows actually arise in practice. Table 8 counts them across the configurations of the two skewed generators (SSCA and G500) and of the non-skewed ER generator introduced in § 4.1, over a sweep that we extend here by one additional *scale* per generator. On G500 and SSCA, nearly every configuration contains a row whose output exceeds the morsel target, and a majority contain one that exceeds the entire L2 capacity. The extremes are far beyond the target, as the widest row of G500 at *scale* 20 and *edgefactor* 16 holds 623.0K nonzeros (9.51 MB,  $4.75\times$  L2), while that of SSCA at the same setting holds 657.1K nonzeros (10.03 MB,  $5.01\times$  L2). We further find that the share of the largest morsel taken by its widest row rises steadily with the *scale*, from 55–90% at *scale* 17 to 85–95% at *scale* 18, and from *scale* 19 upward it reaches 98–100%, i.e., the largest morsel simply is that row. Nearly all of the skewed R-MAT configurations evaluated in § 4.2 were therefore already measured in the presence of oversized rows. The trend moreover runs in one direction only, as raising the *scale*, the *edgefactor*, or both makes these rows wider and more frequent on both skewed generators. Handling them is therefore a requirement for any practical SpGEMM implementation on skewed input, rather than a corner case that it may leave aside<sup>23</sup>.

The non-skewed ER generator makes the contrast explicit, as none of its 20 configurations produces an oversized row at all. Its widest row over the entire sweep holds only 7.0K nonzeros (0.107 MB), about a tenth of the morsel target and a twentieth of the L2 capacity. This width is governed predominantly by the *edgefactor*, as a typical ER output row holds roughly  $(2 \text{ edgefactor})^2$  nonzeros, with the widest staying within a small factor of that, and it stays flat while the *scale* grows from 18 to 21, so enlarging the matrix never brings ER closer to the oversized regime. Oversized rows are therefore a signature of skew rather than of problem size, which is also why AMS has little to recover on ER and merely matches its baseline there (§ 4.2).

## B.2 No Performance Cliff at the Cache Boundary

We now present a comparison of boundary cases, in which matrix dimension and density are held fixed and only the widest row varies due to different skew parameters. Recall that SSCA and G500 emit identical vertex and edge counts at the same *scale* and *edgefactor* and differ only in their R-MAT probabilities (§ 4.1), so at *edgefactor* 2

<sup>23</sup>We note that scaling a matrix far beyond this range may diminish the benefit of AMS, and that such a regime would instead warrant distributed processing for efficient execution [53, 92].

**Table 9: Throughput in GFLOPS at two *scale-edgefactor* settings where the widest output row of G500 crosses a cache boundary while that of SSCA does not. The widest output row is relative to the 2 MB private L2.**

|      |      | widest row        | AMS-HASH | HASH  | speedup |
|------|------|-------------------|----------|-------|---------|
| 18-2 | SSCA | 0.46×L2 (fits)    | 0.612    | 0.591 | 1.04×   |
|      | G500 | 0.55×L2 (exceeds) | 0.760    | 0.599 | 1.27×   |
| 19-2 | SSCA | 0.84×L2 (fits)    | 0.730    | 0.593 | 1.23×   |
|      | G500 | 1.02×L2 (exceeds) | 0.860    | 0.598 | 1.44×   |

G500 is the side whose widest row crosses the boundary. We compare AMS-HASH against its flop-count-based counterpart HASH on these two workloads, and report their respective throughput in Table 9.

We can see that in both settings, the oversized-row side obtains the *larger* speedup, by 0.23 and 0.21, while the HASH baseline stays essentially unchanged (0.591 against 0.599 and 0.593 against 0.598 GFLOPS), so crossing the boundary does not by itself degrade performance. Note that the two settings straddle different boundaries, one at half of the L2 capacity and one at the full L2 capacity, yet the gain is almost the same, so no discontinuity is observed at either boundary.

The reason is that crossing a cache boundary is a property of the workload rather than of the scheduler. Once the working set of a row no longer fits in the cache, the extra misses it incurs are a per-row cost that every row-parallel method pays alike, and AMS can neither add to that cost nor remove it, since it never splits a row in the first place. The flat HASH baseline reports exactly this across each pair. AMS instead removes inter-thread contention and imbalance, whose magnitude grows with skew rather than with the width of any single row. The side whose widest row crosses the boundary is also the more skewed side, so it leaves AMS more to recover rather than less, which is why the speedup rises there instead of falling. We can therefore summarize that simply having rows exceeding a cache boundary does not necessarily hinder performance.

## B.3 Row Width Alone Does Not Govern the Speedup

We now turn to the factors that govern the magnitude of the AMS speedup. In Table 10, we present the speedup of AMS-HASH over HASH for every evaluated configuration of the two skewed generators (SSCA and G500), and report alongside it the widest output row, normalized to the capacity of the 2 MB private L2. The table varies the two parameters independently. Along each row, the *edgefactor*, and hence the density, stays fixed while the *scale* grows, so the widest row keeps growing until it crosses the cache boundaries. Down each column, the *scale* stays fixed while the density grows. The two directions point to different explanations.

The width of the widest row alone does not determine the speedup, since the trend along a row of the table behaves differently at low and high density. At *edgefactor* 8, the speedup decreases as the widest row grows, for example from 1.81× to 1.20× on G500 at that density, and the two generators together give a Spearman correlation of  $\rho = -0.98$  between the widest row and the

**Table 10: AMS-HASH speedup over HASH for every evaluated configuration of the two skewed generators. Each cell reads  $[\times L2, \text{speedup}]$ , where  $\times L2$  is the widest output row relative to the 2 MB L2.  $\rho$  is the Spearman correlation between the widest row and the speedup within each *edgefactor* row, pooled over both generators.**

| <i>edgefactor</i> | G500            |                 |                 |                 | SSCA            |                 |                 |                 | $\rho$ |
|-------------------|-----------------|-----------------|-----------------|-----------------|-----------------|-----------------|-----------------|-----------------|--------|
|                   | <i>scale</i> 17 | <i>scale</i> 18 | <i>scale</i> 19 | <i>scale</i> 20 | <i>scale</i> 17 | <i>scale</i> 18 | <i>scale</i> 19 | <i>scale</i> 20 |        |
| 2                 | 0.29, 1.08×     | 0.55, 1.27×     | 1.02, 1.44×     | 1.89, 1.45×     | 0.25, 0.84×     | 0.46, 1.04×     | 0.84, 1.23×     | 1.51, 1.40×     | +0.93  |
| 4                 | 0.43, 1.54×     | 0.80, 1.63×     | 1.50, 1.47×     | 2.81, 1.22×     | 0.41, 1.22×     | 0.76, 1.42×     | 1.39, 1.53×     | 2.57, 1.43×     | -0.21  |
| 8                 | 0.56, 1.81×     | 1.06, 1.44×     | 2.02, 1.32×     | 3.80, 1.20×     | 0.57, 1.73×     | 1.08, 1.64×     | 2.03, 1.29×     | 3.78, 1.24×     | -0.98  |
| 16                | 0.69, 1.65×     | 1.31, 1.49×     | 2.50, 1.24×     | —               | 0.72, 1.91×     | 1.38, 1.56×     | 2.65, 1.34×     | —               | -0.83  |
| 32                | 0.79, 1.71×     | 1.52, 1.45×     | —               | —               | 0.84, 1.88×     | 1.63, 1.56×     | —               | —               | -0.60  |

— denotes a configuration whose throughput we cannot measure, as materializing its output runs out of memory on our platform. Its widest output row is nonetheless counted in Table 8, which needs only the symbolic phase.

speedup. At *edgefactor* 2, however, the speedup increases instead, with  $\rho = +0.93$ , while *edgefactor* 4 sits in between at  $\rho = -0.21$ , indicating no clear pattern. Pooled over all 34 configurations, these opposing trends cancel at  $\rho = -0.14$ , so the widest row bears no consistent relation to the speedup.

It is noteworthy that the comparison above cannot separate the effect of the widest row from that of the growing total output, since the two increase together. We therefore move on to isolate the two with a complementary control that fixes  $nnz(C)$  instead of the density. Specifically, we pair configurations of the same generator whose  $nnz(C)$  agree within 5% while their widest row differs by at least  $1.3\times$  (e.g., G500 at *scale-edgefactor* 17-8 against 18-4, and SSCA at 18-16 against 19-8), so that each pair carries the same output volume, but packs it into fewer and wider rows on one side. Across all configurations in Table 10, we find six pairs that meet this criterion, and the wider side is the slower one in five of them, by 0.10 to 0.45.<sup>24</sup> The clearest is SSCA at 17-32 against 20-4, whose  $nnz(C)$  agree within 0.1% while their widest rows differ by about threefold, and whose speedup falls from  $1.88\times$  to  $1.43\times$ . We therefore maintain that concentrating the same amount of output into wider rows comes with a smaller benefit for AMS, although it still retains a performance advantage over its plain baseline. The row width is not the sole cause of this loss, since the wider side of each pair is also the sparser one, as we discuss below.

The three quantities at play, namely the output volume, the row width, and the density, cannot be varied independently of one another. At a fixed  $nnz(C)$ , the widest row can differ only by trading the *scale* against the *edgefactor*, so the wider side of every pair above is also the sparser one. This leaves density as the remaining factor to examine, and it turns out to be the one that drives the speedup in a consistent direction. Down each column at *scale* 17 and 18, the speedup grows with the *edgefactor*, for example from  $0.84\times$  to  $1.91\times$  on SSCA and from  $1.08\times$  to  $1.81\times$  on G500 at *scale* 17, before it plateaus at the highest densities. At *scale* 19 and 20 the peak arrives earlier and the speedup declines afterwards, for the reasons we give below, which also lie at the root of the AMS advantage.

As we have demonstrated in § 2.2, AMS saves runtime by removing the inter-thread cache contention that arises during accumulation, so its saving grows with the runtime share that a workload spends on accumulation. A denser workload performs more multiply-accumulate work per output nonzero, i.e., it has a higher compression factor  $cf$ , and therefore spends a larger runtime share on in-cache accumulation. A sparser workload instead spends most of its runtime streaming operands and materializing output, which leaves little contention to remove and too little accumulation to amortize the  $cf$ -based morsel sorting that AMS adds (§ 4.2). AMS therefore shines in the denser cases, where accumulation takes a large share of the runtime. As for the larger *scales*, other costs claim that runtime instead, such as gathers that miss the LLC and the materialization of a growing output. The accumulation share, therefore, stops growing, and raising the *edgefactor* further no longer buys additional speedup.

We now take a closer look at these two costs, namely the gathers that miss the LLC and the materialization of a growing output. At the larger *scales*, the shrinking of the AMS benefit at high density can be easily mistaken for an effect of rows crossing the L2 boundary. We rule this cause out on two grounds. First, the timing of the drops does not match the crossing. At *edgefactor* 8, the widest rows of the two generators are nearly identical at every *scale*, at  $0.56$  against  $0.57$ ,  $1.06$  against  $1.08$ ,  $2.02$  against  $2.03$ , and  $3.80$  against  $3.78$  times the L2, so both cross the boundary at the same step. Their largest drops nonetheless occur at different steps, from *scale* 17 to 18 on G500 but from *scale* 18 to 19 on SSCA. Second, the two costs above grow more significant across the *scales*, and neither of them depends solely on the row width. As the *scale* grows from 17 to 20, the operand matrix  $B$  grows from about 34 MB to 269 MB and no longer fits in the 60 MB LLC, so every gather in Gustavson’s inner loop [50] becomes a random access to DRAM. Meanwhile,  $nnz(C)$  grows by about  $20\times$  over the same range, and materializing this output is a fixed cost that no scheduler can remove. Both effects reduce the runtime share spent on contention, which is the component that AMS optimizes directly.

Nevertheless, AMS retains a clear advantage throughout the oversized regime. Across all 19 configurations whose widest row exceeds the full L2 capacity and whose throughput is measured, AMS-HASH outperforms HASH in every case, with a minimum speedup of  $1.20\times$  and a geometric mean of  $1.40\times$ .

<sup>24</sup>We write each pair as *scale-edgefactor* with the narrower side first and give the speedup of each side in parentheses. The speedup shrinks in five pairs, namely G500 17-8 ( $1.81\times$ ) against 18-4 ( $1.63\times$ ), G500 17-16 ( $1.65\times$ ) against 20-2 ( $1.45\times$ ), G500 19-8 ( $1.32\times$ ) against 20-4 ( $1.22\times$ ), SSCA 18-16 ( $1.56\times$ ) against 19-8 ( $1.29\times$ ), and SSCA 17-32 ( $1.88\times$ ) against 20-4 ( $1.43\times$ ). The sole exception is G500 18-8 ( $1.44\times$ ) against 19-4 ( $1.47\times$ ), whose speedup grows by 0.03 instead.

## B.4 Limitation of Inter-Row Scheduling

The scope of the results above is set by AMS scheduling *between* rows rather than within them. A single row that dominated the total computation would therefore set the critical path for AMS, and equally for every other row-parallel method, since such a row cannot be divided without intra-row parallelism. In our workloads the widest row accounts for at most 0.09% of  $nnz(C)$ , so this regime does not arise. Should it ever arise, intra-row parallelism and segmented accumulation [4, 120] would have to be carefully considered as complementary techniques.

## C THE PERFORMANCE GAP AGAINST MKL

We now discuss the relative performance of AMS-HYB against Intel MKL [61]. The performance margin, i.e., the speedup of AMS-HYB over MKL, is narrower on the relational workloads of § 4.6 than on the R-MAT workloads of § 4.2. We attribute the difference to two factors, neither of which is an absence of skew in relational data. First, the large R-MAT margins, which reach 3.83× over MKL (§ 4.2), arise predominantly from the two skewed generators G500 and SSCA rather than from the non-skewed ER. Their row-wise skew is far more extreme than that of our relational workloads, since the Gini coefficients of G500 and SSCA range from 0.85 to 0.92, whereas the relational workloads, skewed to varying degrees, span 0.13 to 0.72 (Table 3). Second, the narrow margins, i.e., those below 10%, are concentrated on the workloads with very low row and column counts, such as RD23, a 6.72K-square matrix on which the margin is only 1.06×. MKL is a strong baseline in precisely this regime, as it is a heavily optimized industrial library that issues fewer instructions per flop than most methods, including AMS<sup>25</sup>. When the workload is small in size, and especially when its working set stays mostly resident in cache, the SpGEMM kernel is instruction-bound rather than memory-bound, so that instruction advantage translates directly into runtime.

To separate the two factors, we take RD23, one of the workloads on which our margin over MKL is smallest, and scale it up to different degrees with the same R-MAT [24] scaling procedure used for our synthetic workloads<sup>26</sup>. This procedure enlarges the instance while preserving its distribution, so that skew is held fixed and only scale varies. Table 11 reports the runtime and the microarchitectural profile at scaling degrees  $k = 0$  to  $k = 4$ , where  $k$  is the number of scaling steps applied. With a larger  $k$ , the workload is  $2^k$  times larger. We can see from the table that the margin is 1.06× at  $k = 0$  and widens monotonically to 1.60× at  $k = 4$ . MKL’s instruction advantage stays essentially constant as  $k$  grows (15.8 against our 21.0 instructions per flop), so it does not explain the widening margin. The difference comes from memory behavior instead. Over the same range, MKL’s cycles per flop grow by 2.42×, whereas ours grow by only 1.53×. Its stalled cycles rise from 1.34% to 25.65% of all cycles, against 0.39% to 10.23% for ours, and its per-core retire rate falls to 0.42× of its initial value, against 0.68× for ours. These trends

**Table 11: Runtime and microarchitectural profile of AMS-HYB and MKL on RD23 across scale-up levels  $k$ , which enlarge the instance while preserving its distribution. MKL’s instruction advantage stays constant, whereas its memory stalls and its runtime grow faster than ours.**

|                 |         | $k=0$  | $k=1$  | $k=2$  | $k=3$  | $k=4$   | $k_4/k_0$ |
|-----------------|---------|--------|--------|--------|--------|---------|-----------|
| time (s)        | AMS-HYB | 0.856  | 1.815  | 4.093  | 8.957  | 20.883  | 24.4×     |
|                 | MKL     | 0.910  | 2.138  | 5.346  | 12.967 | 33.511  | 36.8×     |
| ins/flop        | AMS-HYB | 21.00  | 21.07  | 21.18  | 21.36  | 21.69   | 1.03×     |
|                 | MKL     | 15.77  | 15.81  | 15.86  | 15.91  | 15.95   | 1.01×     |
| cyc/flop        | AMS-HYB | 15.45  | 16.38  | 18.48  | 20.23  | 23.58   | 1.53×     |
|                 | MKL     | 20.70  | 23.80  | 29.03  | 36.54  | 50.05   | 2.42×     |
| per-core retire | AMS-HYB | 1.36   | 1.29   | 1.15   | 1.06   | 0.92    | 0.68×     |
|                 | MKL     | 1.52   | 1.33   | 1.09   | 0.87   | 0.64    | 0.42×     |
| stall/flop      | AMS-HYB | 0.0595 | 0.1719 | 0.3545 | 0.7688 | 2.4130  | 40.5×     |
|                 | MKL     | 0.2774 | 1.0970 | 3.0742 | 6.3427 | 12.8378 | 46.3×     |
| stall %cyc      | AMS-HYB | 0.39%  | 1.05%  | 1.92%  | 3.80%  | 10.23%  | 26.2×     |
|                 | MKL     | 1.34%  | 4.61%  | 10.59% | 17.36% | 25.65%  | 19.1×     |

indicate that the workload becomes memory-bound as it grows, which is exactly the regime that AMS-HYB targets and where MKL is least effective. Overall, the narrow margins on the relational workloads come from their mild skew and their small scale, not from a limitation of AMS-HYB. Once such a workload is enlarged while its skew is held fixed, the margin of AMS-HYB is restored, which confirms that our framework retains its advantage wherever the workload is large enough for memory behavior to dominate.

<sup>25</sup>MKL is proprietary, so we are not able to apply its instruction-level optimizations to our AMS-based methods.

<sup>26</sup>R-MAT is a recursive Kronecker construction [24, 74], so one scaling step increments the *scale* parameter, i.e., adds one level to the Kronecker recursion and doubles the row and column counts, while the *edgefactor* and the partition probabilities ( $a, b, c, d$ ) fitted to RD23 are held fixed. The degree distribution, and hence the skew, is therefore preserved across the scaling degrees.